Die neusten Features in Java 25 (Teil 1)

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Scoped Values – JEP 506 🌟

Scoped Values erlauben es, einen oder mehrere Werte an eine oder mehrere Methoden zu übergeben, ohne sie als explizite Parameter zu definieren und von einer Methode zur nächsten weiterreichen zu müssen.

Das folgende Beispiel zeigt, wie ein Webserver den eingeloggten User als *Scoped Value* definiert und in dessen Scope die Methode *restAdapter.process(...)* aufruft. Die Aufrufkette führt letztendlich zum Aufruf von *Repository.getData(...)*, und diese Methode kann dann über den *Scoped Value* auf den eingeloggten User zugreifen.

```
public class Server {
  public final static ScopedValue<User>
      LOGGED_IN_USER = ScopedValue.newInstance();
  . . .
  private void serve(Request request) {
    User loggedInUser = authenticateUser(request);
    ScopedValue
        .where(LOGGED_IN_USER, loggedInUser)
        .run(() -> restAdapter.process(request));
  }
  . . .
}
```

```
public class Repository {
  . . .
  public Data getData(UUID id) {
    Data data = findById(id);
    User loggedInUser = Server.LOGGED_IN_USER.get();
    if (loggedInUser.isAdmin()) {
      enrichDataWithAdminInfos(data);
    }
    return data;
  }
  . . .
}
```

Eine ausführliche Beschreibung findest du in https://www.happycoders.eu/de/java/scoped-values/.

Compact Source Files and Instance Main Methods – JEP 512 🌟

Ein Java-Programm kann in Zukunft ohne Klassendeklaration und mit einer nicht-statischen, nicht-öffentlichen, parameterlosen *main*-Methode geschrieben werden.

Für einfache IO-Tasks wurden die Methoden *IO.print()*, *IO.println()* und *IO.readln()* eingeführt. Damit ist der Code rechts ein gültiges, vollständiges Java-Programm.

```
void main() {
  IO.println("Hello, world!");
}
```

Flexible Constructor Bodies – JEP 513 🌟

Bisher war es nicht möglich, vor dem Aufruf des Super-Konstruktors mit *super(...)* oder vor dem Aufruf eines alternativen Konstruktors mit *this(...)* weiteren Code auszuführen. Die Validierungen von Parametern oder die Berechnung von Argumenten für den Aufruf von *super(...)* oder *this(...)* musste daher kompliziert in separate Methoden ausgelagert werden.

Auch dass man Felder in abgeleiteten Klassen erst *nach* dem Aufruf von *super(...)* initialisieren konnte, konnte zu Problemen führen: Wenn der Super-Konstruktor eine Methode aufruft, die in der abgeleiteten Klasse überschrieben wurde und dort auf Felder der abgeleitete Klasse zugreift, dann sind diese Felder zu dem Zeitpunkt des Methodenaufrufs noch uninitialisiert.

Dank JDK Enhancement Proposal 513 ist nun auch Code wie der folgende gültig:

```
public class Shape {
  private final Color color;

  public Shape(Color color) {
    this.color = color;
    printMe();
  }

  void printMe() {
    System.out.println(
        "color = " + color);
  }
}
```

```
public class Rectangle extends Shape {
  private final double width;
  private final double height;

  public Rectangle(Color color, double width, double height) {
    if (width < 0 || height < 0) throw new IllegalArgumentException();
    this.width = width;
    this.height = height;
    super(color);
  }

  void printMe() {
    super.printMe();
    System.out.println("width = " + width + ", height = " + height);
  }
}
```

Hierbei gelten folgende Einschränkungen: Im Code vor *super(...)* oder *this(...)* – dem sogenannten Prolog – darf der Code zwar Felder initialisieren, nicht aber *lesend* auf Felder der Klasse zugreifen und keine nicht-statische Methoden der Klasse aufrufen. Er darf außerdem keine Instanzen von nicht-statischen inneren Klassen erzeugen, da diese dann eine Referenz auf das potentiell uninitialisierte Elternobjekt haben würden.

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 25 (Teil 2)

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Module Import Declarations – JEP 511 ⭐

Seit Java 1.0 werden in jeder *.java*-Datei automatisch alle Klassen des Pakets *java.lang* importiert. Deshalb können wir Klassen wie *Object*, *String*, *Integer*, *Exception*, *Thread*, usw. ohne explizite *import*-Statements verwenden.

Ebenso konnten wir schon immer komplette Pakete importieren. So führt z. B. *import java.util.** dazu, dass wir Klassen wie *List*, *Set*, *Map*, *ArrayList*, *HashSet* und *HashMap* nicht einzeln importieren müssen.

JDK Enhancement Proposal 511 macht es nun möglich, dass wir auch komplette *Module* importieren können. Genauer gesagt: alle Klassen in den vom Modul exportierten Paketen. So können wir z. B. wie folgt das komplette *java.base*-Modul importieren und Klassen dieses Moduls (im Beispiel *List*, *Map*, *Collectors*, *Stream*) ohne weitere Imports nutzen:

```
import module java.base;

public static Map<Character, List<String>> groupByFirstLetter(String... values) {
  return Stream.of(values).collect(
      Collectors.groupingBy(s -> Character.toUpperCase(s.charAt(0))));
}
```

Ist ein Klassenname mehrdeutig, wie z. B. *Date*, dann muss die Klasse direkt importiert werden (also mit *import java.util.Date* oder *import java.sql.Date*). In JShell sowie in impliziert deklarierten Klassen wird das *java.base*-Modul automatisch importiert.

Compact Object Headers – JEP 519 ⭐

Jedes Java-Objekt besitzt einen Objekt-Header, der aktuell 12 Byte groß ist (oder 16 Byte, wenn Compressed Class Pointers deaktiviert sind). In Project Lilliput arbeiten JDK-Entwickler seit Jahren daran, den Header weiter zu komprimieren. In Java 25 können nun „Compact Object Headers“ aktiviert werden, mit denen der Header auf 8 Byte reduziert wird.

Erreicht wurde dies durch die Entfernung ungenutzter Bits und eine Komprimierung des sogenannten „Class Words“ von 32 Bit auf 22 Bit. Dabei werden die Klasseninformationen an durch 1.024 (= 2^{10}) teilbaren Speicheradressen im Metaspace abgelegt, sodass die unteren 10 Bits der Adresse immer 0 sind und somit entfallen können.

Compact Object Headers müssen mit der VM-Option *-XX:+UseCompactObjectHeaders* aktiviert werden, sollen aber in einer der kommenden Java-Versionen standardmäßig aktiviert sein.

Details findest du unter https://www.happycoders.eu/de/java/compact-object-headers/.

Generational Shenandoah – JEP 521 ⭐

Die auf extrem niedrige Pausenzeiten spezialisierten Garbage Collectoren *ZGC* und *Shenandoah* machten ursprünglich keinen Unterschied zwischen alten und neuen Objekten und nutzten somit nicht die „Schwache Generationshypothese“, die besagt, dass die meisten Objekte kurz nach ihrer Entstehung wieder sterben, und dass diejenigen Objekte, die bereits ein gewisses Alter erreicht haben, in der Regel auch noch länger leben werden.

Ein „Generational Garbage Collector“ hingegen teilt den Heap in eine junge und eine alte Generation auf, wodurch sich die Leistung verbessert, da ältere Objekte seltener aufgeräumt werden müssen. In Java 21 wurde ein „Generational Mode“ für ZGC eingeführt, der seit Java 23 standardmäßig aktiviert ist.

In Java 25 ist nun auch der „Generational Mode“ für Shenandoah GC produktionsreif. Dieser kann mit der VM-Option *-XX:ShenandoahGCMode=generational* aktiviert werden.

Ahead-of-Time Command-Line Ergonomics – JEP 514 ⭐

In Java 24 wurde *Ahead-of-Time Class Loading & Linking* vorgestellt. Dabei werden in einer Vorbereitungsphase alle von der Anwendung benötigten Klassen gelesen, geparst, geladen und gelinkt und dann in diesem Zustand in einem Cache gespeichert. Beim Start der Anwendung müssen diese Schritte dann nicht mehr ausgeführt werden; die Anwendung kann auf die geladenen und gelinkten Klassen direkt über den Cache zugreifen.

In Java 24 bestand die Vorbereitungsphase aus zwei Schritten: Im „Record Mode“ analysiert die JVM die Anwendung in einem Trainingslauf und speichert Informationen über die geladenen und gelinkten Klassen in der AOT-Konfiguration. Im „Create Mode“ erzeugt die JVM aus der AOT-Konfiguration den AOT-Cache, aus dem die JVM dann bei jedem weiteren Start der Anwendung die Klassen in geladener und gelinkter Form lädt.

Seit Java 25 können die beiden ersten Schritte durch ein einziges Kommando ausgeführt werden, z. B.:

*java **-XX:AOTCacheOutput=AotTest.aot** -cp AotTest.jar eu.happycoders.AotTest*

Gestartet wird die Anwendung danach wie folgt:

*java **-XX:AOTCache=AotTest.aot** -cp AotTest.jar eu.happycoders.AotTest*

Eine detailliertere Anleitung findest du unter https://www.happycoders.eu/de/java/ahead-of-time-class-loading-and-linking/.

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 25 (Teil 3)

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Ahead-of-Time Method Profiling – JEP 515 ⭐

Die JVM sammelt während der Ausführung einer Java-Anwendung kontinuierlich Daten über aufgerufene Methoden und deren CPU-Verbrauch, um diese Methoden dynamisch zu optimieren und in Maschinencode zu übersetzen. Da dieser Vorgang eine Weile dauert, ist eine Java-Anwendung zu Beginn – in der sogenannten Aufwärmphase („warm-up phase") – langsamer und erreicht ihre volle Leistungsfähigkeit erst nach einigen Sekunden.

Ahead-of-Time Class Loading and Linking (s. vorheriger Abschnitt) erzeugt per Trainingslauf einen AOT-Cache mit geladenen und gelinkten Klassen, was den Start einer Anwendung beschleunigt. *Ahead-of-Time Method Profiling* erweitert dies, indem zusätzlich Methoden-Profile, also die Daten über die CPU-Nutzung der Methoden, in diesem Cache gespeichert werden.

So können beim Start die wichtigsten „Hotspots", also die am häufigsten aufgerufenen Methoden, sofort kompiliert werden. In im Rahmen des JEP durchgeführten Tests konnte dadurch die Startzeit um bis zu 19 % verkürzt werden.

JFR Method Timing & Tracing – JEP 520 ⭐

Beim Profiling liest der Java Flight Recorders (JFR) zu bestimmten Zeitpunkten die Aufrufstacks aller Threads aus und schätzt daraus Häufigkeiten und Dauern von Methodenaufrufen. Dieses Sampling ist jedoch ungenau und liefert keine exakten Werte.

Bisher boten Tools wie *JProfiler*, *YourKit* oder *DataDog* präzisere Messungen, indem sie als Java Agent Profiling-Code in Methoden einfügen – allerdings mit spürbarem Overhead.

Mit JDK Enhancement Proposal 520 kann die JVM nun selbst Aufrufe und Dauern exakt messen und über Filter gezielt Klassen oder Methoden auswählen. Das ermöglicht höhere Genauigkeit als Sampling bei geringerem Overhead.

Key Derivation Function API – JEP 510

Eine Key Derivation Function (KDF, deutsch: Schlüsselableitungsfunktion) ist eine Methode, um aus einem geheimen Wert wie einem Passwort, einer Passphrase oder einem kryptographischen Schlüssel einen oder mehrere neue kryptographische Schlüssel abzuleiten. Damit Sicherheitsanbieter KDF-Algorithmen implementieren und anbieten können und wir sie in Anwendungen einsetzen können, bedarf es einer einheitlichen API.

Wie diese API funktioniert, erfährst du in https://www.happycoders.eu/de/java/java-25-features/.

Weitere Änderungen in Java 25

Remove the 32-bit x86 Port (JEP 503) – Nachdem in Java 24 der 32-Bit-Port für Windows entfernt wurde, wird in Java 25 auch die letzte verbleibende 32-Bit-Variante (die für Linux) vollständig entfernt. So wird der Extraaufwand für das Entwickeln und Testen der 32-Bit-Portierungen eliminiert, und die JDK-Developer können sich voll und ganz auf neue Features fokussieren.

JFR Cooperative Sampling (JEP 518) – Cooperative Sampling ist ein Mechanismus im Java Flight Recorder (JFR), der beim Profiling von Anwendungen zu einer höheren Performance, einer stabileren Ausführung der JVM und einer präziseren Erfassung der Laufzeitstatistiken ohne den sogenannten *Safepoint Bias* führt.

Relax String Creation Requirements in *StringBuilder* and *StringBuffer* – Die Spezifikationen der *substring()*-, *subSequence()*- und *toString()*-Methoden der Klassen *StringBuilder* und *StringBuffer* erforderten bisher, dass immer ein neu erstelltes *String*-Objekt zurückgegeben wird. Diese Anforderung wurde aus der Spezifikation entfernt, so dass diese Methoden nun z. B. für einen leeren String eine ""-Konstante zurückgeben können, was schneller ist, als einen *neuen* leeren String zu erzeugen.

New Methods on *BodyHandlers* and *BodySubscribers* To Limit The Number of Response Body Bytes Accepted By The *HttpClient* – Die Klassen *java.net.http.HttpResponse.BodyHandlers* und *java.net.http.HttpResponse.BodySubsribers* wurden um je eine *limiting()*-Methode erweitert, mit der die Größe einer Antwort auf eine HTTP-Anfrage begrenzt werden kann.

The *UseCompressedClassPointers* Option is Deprecated – Der Support für unkomprimierte Klassenpointer soll in einer zukünftigen Java-Version entfernt werden. Entsprechend wurde die VM-Option *UseCompressedClassPointers* als *deprecated* markiert. Mehr erfährst du unter https://www.happycoders.eu/de/java/object-headers-compressed-class-pointers/.

Various Permission Classes Deprecated for Removal – In Java 17 wurde der Security Manager *deprecated for removal* gekennzeichnet. Seit Java 24 kann der Security Manager nicht mehr aktiviert werden. In Java 25 wurden nun zahlreiche ...*Permission*-Klassen, die nur mit dem Security Manager verwendbar waren, ebenfalls als *deprecated for removal* markiert.

Syntax Highlighting for Code Fragments – Das *javadoc*-Tool wurde um die Kommandozeilenoption *--syntax-highlight* erweitert. Wird diese beim Aufruf des *javadoc*-Kommandos angegeben, wird die Library *Highlight.js* in die generierte Dokumentation eingebunden und der Code in *{@snippet}*-Tags und HTML-Elementen entsprechend farblich gekennzeichnet.

Add Standard System Property *stdin.encoding* – Über die neue System Property *stdin.encoding* kann der Zeichensatz für das Lesen von *System.in* festgelegt werden. Wenn nicht explizit angegeben – z. B. durch *-Dstdin.encoding=UTF-8* –, wird der Wert durch das Betriebssystem und die Benutzerumgebung ermittelt.

 Copyright © 2025 HappyCoders.eu

Bonus: Preview-Features in Java 25 (Teil 1)

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Stable Values (Preview) – JEP 502 🌟

Stable Values ermöglichen die saubere, performante und threadsichere Initialisierung von Werten, die nicht beim Start einer Anwendung gesetzt werden sollen (oder können), sondern erst beim ersten Zugriff darauf („lazy initialization"). Bisher war das nur durch das aufwändige, fehleranfällige und nicht durch die JVM optimierbare Double-Checked Locking Idiom threadsicher möglich – bzw. bei statischen Werten auch mit dem Initialization-on-Demand Holder Idiom.

Ein *Stable Value* lässt sich z. B. wie folgt definieren:

```
private final Supplier<Settings> settings = StableValue.supplier(this::loadSettingsFromDatabase);
```

Der erste Aufruf von *settings.get()* führt dann zum Aufruf von *loadSettingsFromDatabase()* und dem Speichern des erhaltenen Objekts. Alle Folgeaufrufe liefern das gespeicherte Objekt zurück. Die JVM kann diese Folgeaufrufe genau wie Zugriffe auf Konstanten optimieren (u. a. durch „constant folding").

Mit *StableValue.list()* lässt sich eine Liste definieren, deren Elemente bei Zugriff initialisiert und danach eingefroren werden:

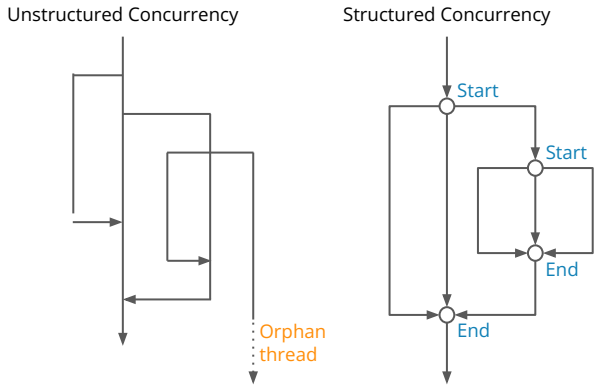
```
List<Double> squareRoots = StableValue.list(100, Math::sqrt);
```

Erst beim ersten Zugriff auf ein Listenelement – sei es mit *first()*, *get(int index)*, *last()* oder bei einer Iteration – wird dieses berechnet. Danach bleibt es konstant, und weitere Zugriffe darauf können wieder durch die JVM optimiert werden.

Neben *Stable Values* und *Lists* gibt es auch *Stable Maps*, *Stable Functions* und *Stable IntFunctions*. Die vollständige *Stable Values* API findest du unter https://www.happycoders.eu/de/java/stable-values/.

Structured Concurrency (Fifth Preview) – JEP 505 🌟

Structured Concurrency ermöglicht die einfache Koordination von nebenläufigen Aufgaben und zeichnet sich durch klar im Code ersichtliche Start- und Endpunkte nebenläufiger Subtasks aus – wohingegen bei *Unstructured Concurrency* Start und Ende von Subtasks deutlich schwerer im Code erkennbar sind (s. Grafik rechts).



JEP 505 führt mit *StructuredTaskScope* eine Kontrollstruktur ein, die den Anfang und das Ende nebenläufiger Tasks klar definiert, eine saubere Fehlerbehandlung ermöglicht und Teilaufgaben, deren Ergebnisse nicht mehr benötigt werden, geordnet abbrechen kann.

Im folgenden Beispiel laden wir gleichzeitig eine Bestellung, einen Kunden und eine Rechnungsvorlage und erstellen daraus eine Rechnung. Sobald eine der Teilaufgaben fehlschlägt, werden die noch laufenden Subtasks beendet, und der Fehler wird an den Aufrufer propagiert:

```
Invoice createInvoice(int orderId, int customerId, String language)
    throws InterruptedException, ExecutionException {

  try (var scope = StructuredTaskScope.open()) {
    Subtask<Order>           orderSubtask    = scope.fork(() -> orderService.getOrder(orderId));
    Subtask<Customer>        customerSubtask = scope.fork(() -> customerService.getCustomer(customerId));
    Subtask<InvoiceTemplate> templateSubtask = scope.fork(() -> templateService.getTemplate(language));

    scope.join();

    var order    = orderSubtask.get();
    var customer = customerSubtask.get();
    var template = templateSubtask.get();

    return Invoice.from(order, customer, template);
  }
}
```

Ebenso einfach können wir eine *race()*-Methode implementieren, die zwei Tasks startet, das Ergebnis des zuerst beendeten Tasks zurückliefert und den zweiten Task abbricht (s. Beispiel rechts).

Eine ausführliche Beschreibung findest du in https://www.happycoders.eu/de/java/structured-concurrency/.

```
public static <R> R race(Callable<R> task1, Callable<R> task2)
    throws InterruptedException {

  try (var scope = StructuredTaskScope.open(
      Joiner.<R>anySuccessfulResultOrThrow())) {

    scope.fork(task1);
    scope.fork(task2);

    return scope.join();
  }
}
```

 Copyright © 2025 HappyCoders.eu

Bonus: Preview-Features in Java 25 (Teil 2)

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Primitive Types in Patterns, instanceof, and switch (3rd Preview) – JEP 507 ⭐

Mit *instanceof* und *switch* können wir seit Java 16 („Pattern Matching for instanceof") bzw. Java 21 („Pattern Matching for switch") überprüfen, ob ein Objekt von einem bestimmten Typ ist, und wenn ja, dieses Objekt an eine Variable dieses Typs binden, einen bestimmten Programmpfad ausführen und in diesem Programmpfad die neue Variable benutzen.

Durch JDK Enhancement Proposal 507 dürfen nun auch alle *primitiven* Typen im Pattern Matching verwendet werden. Allerdings ist die genaue Funktionsweise eine andere als bei der Verwendung von Objekten – denn bei primitiven Typen gibt es ja keine Vererbung:

Sei *a* eine *Variable* eines primitiven Typen (also *byte*, *short*, *int*, *long*, *float*, *double*, *char* oder *boolean*) und *B* einer eben dieser primitiven *Typen*. Dann ergibt *a instanceof B* genau dann *true*, wenn der präzise Wert von *a* auch in einer Variablen vom Typ *B* gespeichert werden kann. Hier ein Beispiel:

```
int value = ...
if (value instanceof byte b) {
  System.out.println("b = " + b);
}
```

Der Code ist wie folgt zu lesen: Wenn der Wert der Variablen *value* auch in einer *byte*-Variablen gespeichert werden kann, dann weise der *byte*-Variablen *b* diesen Wert zu und gebe diesen aus. Für *value = 5* wäre das z. B. der Fall, für *value = 1000* hingegen nicht, da *byte* lediglich Werte von -128 bis 127 speichern kann.

Genau wie bei Objekten darfst du auch bei primitiven Typen direkt im *instanceof*-Check mit *&&* weitere Prüfungen anschließen. Der folgende Code z. B. gibt nur positive *byte*-Werte (also 1 bis 127) aus:

```
if (value instanceof byte b && b > 0) {
  System.out.println("b = " + b);
}
```

Besonders interessant wird es beim Matching von und gegen Gleitkommazahlentypen – mehr dazu findest du unter https://www.happycoders.eu/de/java/java-25-features/.

Vector API (Tenth Incubator) – JEP 508

Die *Vector API* wird es ermöglichen, Vektor-Operationen wie die folgende auf möglichst effiziente Art und Weise auf die Vektor-Instruktionen moderner CPUs abzubilden. So können solche Berechnungen in nur wenigen CPU-Zyklen (bzw. bis zu einer bestimmten Vektorgröße in nur einem einzigen CPU-Zyklus) durchgeführt werden:

$$\begin{bmatrix} 7 \\ 13 \\ 8 \end{bmatrix} + \begin{bmatrix} 16 \\ 3 \\ 6 \end{bmatrix} = \begin{bmatrix} 7+16 \\ 13+3 \\ 8+6 \end{bmatrix} = \begin{bmatrix} 23 \\ 16 \\ 14 \end{bmatrix}$$

Da sich die Vector API noch im Incubator-Stadium befindet, also noch wesentlichen Änderungen unterliegen kann, zeige ich an dieser Stelle noch keinen Code.

JFR CPU-Time Profiling (Experimental) – JEP 509

Beim „Profiling" erzeugt der Java Flight Recorder Statistiken darüber, welche Methoden wie viel Zeit in Anspruch nehmen. Dabei wird bisher die sogenannte *Ausführungszeit* gemessen, also diejenige Zeit, die vom Eintritt in die Methode bis zum Austritt vergangen ist. Diese Zeit ist unabhängig von der CPU-Nutzung. So kann eine Methode, die eine Sekunde rechnet, dieselbe Ausführungszeit haben wie eine, die eine Sekunde auf eine Datenbankantwort wartet.

Durch die neue Option *jdk.CPUTimeSample#enabled=true* lässt sich nun (zunächst nur auf Linux) auch das sogenannte *CPU-Zeit*-Profiling aktivieren, um nicht die Ausführungszeit, sondern die Zeit der CPU-Nutzung zu messen:

java -XX:StartFlightRecording=jdk.CPUTimeSample#enabled=true,filename=profile.jfr ...

PEM Encodings of Cryptographic Objects (Preview) – JEP 470

Bisher war es extrem mühevoll, Schlüssel oder Zertifikate im PEM-Format (s. rechts) zu importieren oder exportieren. Für das Dekodieren eines verschlüsselten Private Keys im PEM-Format beispielsweise waren über ein Dutzend Code-Zeilen nötig.

In Java 25 (mit aktivierten Preview-Features) ist das nun mit wenigen Zeilen möglich:

```
PrivateKey privateKey = PEMDecoder.of()
   .withDecryption(passphrase.toCharArray())
   .decode(encryptedPrivateKeyPemEncoded, PrivateKey.class);
```


```
-----BEGIN CERTIFICATE-----
MIIDtzCCAz2gAwIBAgISBUCeYELtjMmr4FAIqHapebbFMAoGCCq
CzAJBgNVBAYTAlVTMRYwFAYDVQQKEw1MZXQncyBFbmNyeXB0MQs
. . .
DBeMde1YpWNXpF9+B/OMKgn7RgXRj5b2QpBCnFsP92T4cK/Nn+x
toSQBlYnNHv0eXnWkI8TmXsU/A6rU4Gxdr9GbGixgRJvk==
-----END CERTIFICATE-----
```


 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 24 (Teil 1)

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Stream Gatherers – JEP 485 🌟

Die Java-Stream-API hat mit *filter*, *map*, *flatMap*, *mapMulti*, *distinct*, *sorted*, *peak*, *limit*, *skip*, *takeWhile* und *dropWhile* nur eine begrenzte Auswahl intermediärer Stream-Operationen. Alternative Operationen, wie *window* und *fold*, sucht man vergeblich.

Anstatt den vielseitigen Wünschen der Community nachzugehen und alle gewünschten Methoden ins JDK zu integrieren, entschieden sich die JDK-Developer, stattdessen eine API zu entwickeln, die es sowohl den JDK-Developern selbst als auch der Java-Community ermöglicht, beliebige intermediäre Stream-Operationen zu schreiben. Mit JEP 485 wird zum einen die neue API ausgeliefert und zum anderen eine ganze Reihe vordefinierter Gatherer, so auch die geforderten *window*- und *fold*-Operationen.

Mit den *Fixed*- und *Sliding-Window*-Operation z. B. kannst du Stream-Elemente in Listen vorgegebener Größen gruppieren:

```
List<String> words = List.of("the", "be", "two", "of", "and", "a", "in", "that");
List<List<String>> fixedWindows   = words.stream().gather(Gatherers.windowFixed(3)).toList();
List<List<String>> slidingWindows = words.stream().gather(Gatherers.windowSliding(3)).toList();
System.out.println(fixedWindows);
System.out.println(slidingWindows);
```

Dieses kleine Beispielprogramm gibt folgendes aus:

```
[[the, be, two], [of, and, a], [in, that]]
[[the, be, two], [be, two, of], [two, of, and], [of, and, a], [and, a, in], [a, in, that]]
```

Welche Stream Gatherer das JDK noch bereitstellt und wie du eigene Gatherer für intermediäre Operationen selbst schreiben kannst, erfährst du in https://www.happycoders.eu/de/java/stream-gatherers/.

Synchronize Virtual Threads without Pinning – JEP 491 🌟

Seit Java 21 wurden virtuelle Threads beim Aufruf von blockierendem Code innerhalb eines *synchronized*-Blocks an ihren Carrier-Thread „gepinnt“, sodass dieser blockiert wurde und keine anderen virtuellen Threads ausführen konnte. Das konnte zu schwerwiegenden Performance-Problemen führen.

Ab Java 24 ist dieses Problem behoben: Virtuelle Threads werden nun während der Synchronisation vom Carrier Thread entkoppelt, sodass dieser weiterhin andere virtuelle Threads bedienen kann.

Warum virtuelle Threads überhaupt gepinnt werden mussten und warum es jetzt auch ohne Pinning funktioniert, erfährst du in https://www.happycoders.eu/de/java/java-24-features/#Synchronize_Virtual_Threads_without_Pinning_JEP_491.

Ahead-of-Time Class Loading & Linking – JEP 483 ⭐

Beim Start einer Java-Anwendung müssen zahllose Klassen gelesen, geparst, geladen und gelinkt werden. Besonders bei großen Backend-Anwendungen kann das zu langen Startzeiten führen.

Innerhalb von Project Leyden wird seit langem an Lösungen dafür gearbeitet, möglichst viele dieser vorbereitenden Aufgaben bereits *vor* dem Start einer Anwendung auszuführen. Durch JDK Enhancement Proposal 483 wird in Java 24 die erste dieser Lösungen vorgestellt: Ahead-of-Time Class Loading & Linking.

Dabei werden in einer Vorbereitungsphase alle von der Anwendung benötigten Klassen gelesen, geparst, geladen und gelinkt und dann in diesem Zustand in einem Cache gespeichert. Beim Start der Anwendung müssen diese Schritte dann nicht mehr ausgeführt werden; die Anwendung kann auf die geladenen und gelinkten Klassen direkt über den Cache zugreifen.

Wie das genau funktioniert, eine Schritt-für-Schritt-Anleitung zum Ausprobieren und einen Vergleich mit AppCDS findest du unter https://www.happycoders.eu/de/java/ahead-of-time-class-loading-and-linking/.

Warn upon Use of Memory-Access Methods in sun.misc.Unsafe – JEP 498

Die mit Java 1.4 eingeführte Klasse *sun.misc.Unsafe* ermöglicht schnellen, direkten Speicherzugriff, ist aber auch riskant. Obwohl sie nie für die direkte Nutzung durch EntwicklerInnen gedacht war, wurde sie mangels Alternativen häufig verwendet. Heute stehen mit *VarHandles* (seit Java 9) und der *Foreign Function & Memory API* (seit Java 22) sichere Alternativen zur Verfügung.

Daher wird *sun.misc.Unsafe* schrittweise abgeschafft: In Java 23 wurden die betroffenen Methoden als *deprecated for removal* markiert, Java 24 gibt Warnungen zur Laufzeit aus, ab voraussichtlich Java 26 wird eine *UnsupportedOperationException* geworfen, und in einer späteren Version werden die Methoden endgültig entfernt werden.

Die VM-Option *--sun-misc-unsafe-memory-access* erlaubt es vorübergehend, Warnungen zu deaktivieren (*allow*), weitere Details auszugeben (*debug*) oder aber auch jetzt schon eine *UnsupportedOperationException* auszulösen (*deny*).

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 24 (Teil 2)

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Late Barrier Expansion for G1 – JEP 475

In der Garbage Collection bezeichnet eine „Barrier“ zusätzlichen Code, der vor oder nach dem Zugriff auf Objekte ausgeführt wird. Beispielsweise helfen Write Barriers dem Garbage Collector, Referenzen zwischen Speicherbereichen nachzuverfolgen, während Read Barriers sicherstellen, dass verschobene Objekte korrekt adressiert werden.

Diese Barriers werden beim Laden eines Java-Programms automatisch in den Maschinencode eingefügt. Der Begriff „Expansion“ bezieht sich darauf, dass der kompakte Bytecode einer Java-Anwendung durch Optimierungen wie Inlining oder Loop Unrolling in speicherintensiveren Maschinencode umgewandelt wird.

Bisher setzte der G1-Garbage Collector auf „Early Barrier Expansion“, bei der Barrier-Code in einer plattformunabhängigen „Intermediate Representation“ (IR) mit dem Anwendungscode kombiniert und dann gemeinsam mit dem Anwendungscode optimiert wurde. Dies hatte den Vorteil einer einheitlichen „Intermediate Representation“, führte jedoch zu längeren Compilierungszeiten und schwer reproduzierbaren Optimierungsfehlern.

Mit JEP 475 wurde daher „Late Barrier Expansion“ eingeführt: Der Barrier-Code liegt nun direkt als optimierter Maschinencode vor und wird erst *nach* der Anwendungscode-Optimierung eingefügt. Dadurch sinkt der Kompilieraufwand, und Anwendungen laufen um etwa 10–20 % schneller. Late Barrier Expansion wird bereits seit Java 15 im ZGC verwendet.

ZGC: Remove the Non-Generational Mode – JEP 490

In Java 21 wurde der „Generational Mode” für den Z Garbage Collector (ZGC) eingeführt. Dieser unterteilt Objekte in eine junge und eine alte Generation, um die Speicherbereinigung zu optimieren. Seit Java 23 war dieser Modus standardmäßig aktiviert, konnte aber durch die VM-Option *-XX:+UseZGC -XX:-ZGenerational* deaktiviert werden.

Mit Java 24 wird der „Non-Generational Mode“ entfernt, um die Wartung des GCs zu vereinfachen. Die Option *-XX:-ZGenerational* hat keinen Effekt mehr und erzeugt eine Warnung; in einer zukünftigen Version wird sie vollständig entfernt werden.

Linking Run-Time Images without JMODs – JEP 493

Eine JDK-Installation besteht aus einem Laufzeit-Image und einem Satz von Java-Modul-Dateien im *jmod*-Verzeichnis. Während das Laufzeit-Image in der Datei *lib/modules* zur Laufzeit einer Java-Anwendung genutzt wird, dienen die Modul-Dateien im *jmod*-Verzeichnis dazu, mit dem *jlink*-Tool benutzerdefinierte Laufzeit-Images zu erstellen.

In Zukunft wird es möglich sein, ein JDK ohne *jmod*-Dateien zu erstellen. Dadurch kann die Größe eines JDKs um etwa 25 % reduziert werden, was besonders im Cloud-Umfeld von Vorteil ist. Die Option ist zunächst deaktiviert, doch in einer zukünftigen Java-Version könnte sie standardmäßig aktiviert werden.

Weitere Änderungen in Java 24

Class-File API (JEP 484) – Die Class-File API ist eine Schnittstelle zum Lesen und Schreiben von *.class*-Dateien, also von kompiliertem Java-Bytecode. Die neue API soll das im JDK intensiv genutzte Bytecode-Manipulations-Framework ASM ablösen.

Quantum-Resistant Module-Lattice-Based Key Encapsulation Mechanism (JEP 496) – Zukünftige Quantencomputer stellen eine Gefahr für die traditionellen kryptografischen Algorithmen wie RSA und Diffie-Hellman dar. Der Einsatz von ML-KEM (Module-Lattice-Based Key Encapsulation Mechanism) soll es in Zukunft ermöglichen, Schlüssel auch im Zeitalter von Quantencomputern sicher auszutauschen. Ein Beispiel dazu findest du im Online-Artikel zu Java 24.

Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm (JEP 497) – Analog zu dem im vorherigen Abschnitt beschriebenen ML-KEM-Verfahren wird das ebenfalls quantensichere ML-DSA-Verfahren (Module-Lattice-Based Digital Signature Algorithm) ins JDK aufgenommen. Auch hierfür findest du ein Beispiel im Java-24-Artikel.

Permanently Disable the Security Manager (JEP 486) – Der ursprünglich für die Absicherung von Java Applets entwickelte – und für heutige Java-Anwendungen nahezu irrelevante „Security Manager” wurde in Java 17 als „deprecated for removal” markiert. Dadurch sollten die Resourcen, die für die Wartung des Security Managers aufgebracht wurden, wichtigeren Projekten zugewiesen werden. In Java 24 kann der Security Manager nicht mehr aktiviert werden, weder beim Start einer Anwendung noch während der Laufzeit. Der Versuch führt zu einer Fehlermeldung.

Remove the Windows 32-bit x86 Port (JEP 479) – Da es kaum noch Bedarf für diese Portierung gibt und die Wartung aufwendig ist (virtuelle Threads z. B. wurden hier gar nicht erst implementiert), wurde diese Portierung vollständig entfernt.

Deprecate the 32-bit x86 Port for Removal (JEP 501) – Mit der Entfernung der 32-Bit-Portierung für Windows ist die 32-Bit-Portierung für Linux die letzte verbleibende 32-Bit-Portierung – und damit die letzte Portierung, für die die JDK-Entwickler Fallbacks für die 32-Bit-Architektur implementieren müssen. Um diesen Extraaufwand vollständig zu eliminieren, wird die Linux-32-Bit-Portierung als „deprecated for removal” markiert.

Deprecate LockingMode Option, along with LM_LEGACY and LM_MONITOR – In Java 21 wurde der neue „Lightweight Locking“-Modus für das Objekt-Monitor-Locking eingeführt, ab Java 23 wurde Lightweight Locking zum Standardmodus. Mit Java 24 wird die Option -XX:LockingMode, mit der der „Legacy“-Modus reaktiviert werden konnten, als „deprecated“ markiert.

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 23

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Markdown Documentation Comments – JEP 467 🌟

Um JavaDoc-Kommentare zu formatieren, mussten wir seit jeher HTML verwenden. JDK Enhancement Proposal 467 macht es möglich, dass wir ab Java 23 JavaDoc-Kommentare zukünftig auch in Markdown schreiben können.

Markdown-Kommentare müssen mit drei Schrägstrichen eingeleitet werden. Das folgende Beispiel zeigt, wie der JavaDoc-Kommentar der Methode *Math.ceilMod(...)* mit Markdown geschrieben werden kann:

```
/// Returns the ceiling modulus of the `long` and `int` arguments.
///
/// The ceiling modulus is `r = x - (ceilDiv(x, y) * y)`,
/// has the opposite sign as the divisor `y` or is zero, and
/// is in the range of `-abs(y) < r < +abs(y)`.
///
/// The relationship between `ceilDiv` and `ceilMod` is such that:
///
/// - `ceilDiv(x, y) * y + ceilMod(x, y) == x`
///
/// For examples, see [#ceilMod(int, int)].
///
/// @param x the dividend
/// @param y the divisor
/// @return the ceiling modulus `x - (ceilDiv(x, y) * y)`
/// @throws ArithmeticException if the divisor `y` is zero
/// @see #ceilDiv(long, int)
/// @since 18
```

Das Beispiel verwendet:

- `...`, um Quellcode zu formatieren,
- eine Leerzeile für vertikalen Abstand,
- ein Bindestrich für eine Aufzählung
- und eckige Klammern für Links.

Außerdem kannst du verwenden:

- **...** für fett,
- *...* oder _..._ für kursiv,
- ``` oder ~~~ für Code-Blöcke,
- Einrückung für Code-Blöcke,
- „1.” für numerierten Listen

JavaDoc-Tags wie *@param*, *@throws* etc. können weiter verwendet werden. Innerhalb von Code oder Code- Blöcken werden sie nicht ausgewertet, sondern unverändert abgedruckt.

ZGC: Generational Mode by Default – JEP 474 ⭐

In Java 21 wurde der „Generational Mode” des Z Garbage Collectors (ZGC) eingeführt. In diesem Modus macht sich der ZGC die „Weak Generational Hypothesis” zu Nutze und legt neue und alte Objekte in zwei separaten Bereichen ab: der „jungen” und der „alten Generation”. Die junge Generation enthält hauptsächlich kurzlebige Objekte und wird häufiger bereinigt, während die alte Generation langlebige Objekte enthält und seltener aufgeräumt werden muss.

Seit Java 21 konnte der Generational Mode mit der VM Option *-XX:+UseZGC -XX:+ZGenerational* aktiviert werden.

Da der Generational Mode für die meisten Anwendungsfälle zu beachtlichen Leistungssteigerungen führt, ist er in Java 23 bei Aktivierung des ZGC durch *-XX:+UseZGC* per default aktiviert. Er kann durch *-XX:+UseZGC -XX:-ZGenerational* deaktiviert werden.

Support for Duration Until Another Instant ⭐

Um die Dauer zwischen zwei *Instant*-Objekten zu bestimmen, musste man bisher *Duration.between(...)* verwenden. Da Java Developer eine solche Funktion intuitiv in *Instant* suchen, wurden *Instant* um die Methode *until(...)* erweitert:

```
Instant now = Instant.now();
Instant later = Instant.now().plus(ThreadLocalRandom.current().nextInt(), SECONDS);
Duration duration = now.until(later);
```

Deprecate the Memory-Access Methods in sun.misc.Unsafe for Removal – JEP 471

Die in *sun.misc.Unsafe* angebotenen Methoden zum Zugriff auf den Speicher waren ursprünglich nur für die interne Nutzung innerhalb des JDK gedacht. Doch mangels Alternativen wurden sie auch ausgiebig außerhalb des JDKs genutzt.

Heute gibt es stabile Alternativen: In Java 9 wurden *VarHandles* eingeführt und in Java 22 die *Foreign Function & Memory API*.

Daher wurden durch JDK Enhancement Proposal 471 alle *sun.misc.Unsafe*-Methoden zum Speicherzugriff als *deprecated for removal* markiert. Eine Auflistung der betroffenen Methoden mitsamt der jeweiligen Alternative findest du im JEP.

Weitere Änderungen in Java 23

Console Methods With Explicit Locale – Die Methoden *Console.printf(...)*, *format(...)*, *readLine(...)* und *readPassword(...)* wurden mit einer Variante mit zusätzlichem *Locale*-Parameter überladen, so dass z. B. Zahlen länderspezifisch formatiert werden können.

Thread.suspend/resume, ThreadGroup.suspend/resume/stop are removed – Diese unsicheren und bereits in Java 1.2 als *deprecated* markierten Methoden wurden im Laufe der Jahre zunächst als *deprecated for removal* markiert, und sie werfen in neueren Java-Versionen eine *UnsupportedOperationException*. In Java 23 wurden sie schließlich entfernt.

Removal of Module jdk.random – Alle Klassen des Moduls *jdk.random* wurden in das *java.base*-Modul verschoben. Solltest du das Java Modulsystem verwenden und irgendwo *requires jdk.random* angegeben haben, dann kannst du diesen Eintrag in Java 23 entfernen (das *java.base*-Modul wird automatisch eingebunden).

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 22

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Unnamed Variables & Patterns – JEP 456 🌟

Ab Java 22 kannst du den Unterstrich „_" benutzen, um Variablen, Pattern-Variablen und ganze Patterns zu kennzeichnen, die du nicht verwenden wirst. Zum Beispiel in Exceptions, Lambdas und beim Pattern Matching:

```
try {
  number = Integer.parseInt(string);
} catch (NumberFormatException _) {
  System.err.println("Not a number");
}
```

```
map
    .computeIfAbsent(key, _ -> new ArrayList<>())
    .add(value);
```

```
if (object instanceof Path(Position(int x1, int y1), _)) {
  System.out.printf("object is a path starting at x = %d, y = %d%n", x1, y1);
}
```

```
switch (object) {
  case Byte _, Short _, Integer _, Long _ -> System.out.println("Integer number");
  case Float _, Double _                  -> System.out.println("Floating point number");
  default                                 -> System.out.println("Not a number");
}
```

Launch Multi-File Source-Code Programs – JEP 458 🌟

Seit Java 11 können wir Java-Programme, die aus nur einer Datei bestehen, direkt ausführen, ohne sie vorher explizit mit *javac* kompilieren zu müssen. In Java 22 wurde diese Fähigkeit erweitert, so dass nun auch Programme, die aus mehreren Dateien bestehen, ohne vorherige Kompilierung gestartet werden können.

Speichere zum Beispiel die folgenden zwei Klassen in jeweils einer Datei, *Hello.java* und *Greetings.java*:

```
public class Hello {
 public static void main(String[] args) {
   System.out.println(Greetings.greet(args[0]));
 }
}
```

```
public class Greetings {
 public static String greet(String name) {
   return "Hello %s!%n".formatted(name);
 }
}
```

Du kannst das Programm dann wie folgt starten, ohne es zuvor mit *javac* zu kompilieren:

```
$ java Hello.java World
Hello World!
```

Foreign Function & Memory API – JEP 454 ⭐

Die Foreign Function & Memory API (oder kurz: FFM API) ermöglicht es, aus Java heraus auf Code außerhalb der JVM (z. B. auf Funktionen in Bibliotheken, die in anderen Programmiersprachen geschrieben wurden) sowie auf nativen Speicher (also Speicher, der nicht von der JVM im Heap verwaltet wird) zuzugreifen.

Im Beispiel rechts wird die *strlen()*-Funktion der Standard-C-Library aufgerufen, um die Länge des Strings „Happy Coding!" zu bestimmen.

```
Linker linker = Linker.nativeLinker();
SymbolLookup stdlib = linker.defaultLookup();

MethodHandle strlen = linker.downcallHandle(
   stdlib.find("strlen").orElseThrow(),
   FunctionDescriptor.of(JAVA_LONG, ADDRESS));

try (Arena offHeap = Arena.ofConfined()) {
  MemorySegment str = offHeap.allocateFrom("Happy Coding!");
  long len = (long) strlen.invoke(str);
  System.out.println("len = " + len);
}
```

Eine ausführliche Beschreibung findest du in https://www.happycoders.eu/de/java/foreign-function-memory-api/.

Region Pinning for G1 – JEP 423

Wenn im Rahmen von JNI (Java Native Interface) mit Speicheradressen von Java-Objekten gearbeitet wird, dürfen diese Objekte vom Garbage Collector nicht an eine andere Position im Speicher verschoben werden. Da der G1 Garbage Collector bisher keine einzelnen Objekte vor dem Verschieben schützen konnte, wurde der Garbage Collector vollständig pausiert, bis diese Speicheradressen wieder freigegeben wurden. Je nach Verhalten einer Anwendung konnte das drastische Konsequenzen auf den Speicherverbrauch und die Performance haben.

Dank des in Java 22 hinzugekommenen „Region Pinnings" kann der G1GC nun eine Speicherregion, die solch ein Objekt enthält, „anheften" – also die darin enthaltenen Objekte nicht an eine andere Speicheradresse verschieben, so dass der Garbage Collector nicht mehr vollständig pausiert werden muss.

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 21 (Teil 1)

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.
Die nicht finalisierten JDK Enhancement Proposals aus Java 21 findest du jetzt auf den Seite 2 und 3.

Mit der Finalisierung von *virtuellen Threads*, *Pattern Matching for Switch* und *Record Patterns* ist das Long-Term Support (LTS) Release Java 21 schon jetzt ein Meilenstein der Java-Geschichte. Diese und die anderen, spannende Java-21-Features konnte ich nicht auf nur einer Seite zusammenzufassen. Ich habe sie daher auf zwei Seiten aufgeteilt.

Virtual Threads – JEP 444 🌟

Bei der Skalierung von Server-Anwendungen stellen Threads oft einen Engpass dar. Ihre Anzahl ist begrenzt, und sie müssen häufig auf Ereignisse warten, wie beispielsweise die Antwort einer Datenbankabfrage oder eines Remote-Aufrufs, oder sie werden durch Locks blockiert.

Bisherige Lösungsansätze wie *CompletableFuture* oder reaktive Frameworks führen zu schwer les- und wartbaren Code, wie z. B.:

```
@GET @Path("/product/{productId}")
public CompletionStage<Response> getProduct(@PathParam("productId") String productId) {
  return productService.getProductAsync(productId).thenCompose(product -> {
      if (product.isEmpty()) {
        return CompletableFuture.completedFuture(Response.status(NOT_FOUND).build());
      }
      return warehouseService
          .isAvailableAsync(productId)
          .thenCompose(available -> available
              ? CompletableFuture.completedFuture(0)
              : supplierService.getDeliveryTimeAsync(product.get().supplier(), productId))
          .thenApply(daysUntilShippable ->
              Response.ok(new ProductPageResponse(product.get(), daysUntilShippable)).build());
      });
}
```

Im Gegensatz zu asynchronem Code wie dem oben ermöglichen virtuelle Threads die Programmierung im herkömmlichen, vertrauten, imperativen Thread-pro-Request-Stil (die *@RunOnVirtualThread*-Annotation ist in Jakarta EE 11 definiert):

```
@GET @Path("/product/{productId}")
@RunOnVirtualThread
public ProductPageResponse getProduct(@PathParam("productId") String productId) {
  Product product = productService.getProduct(productId).orElseThrow(NotFoundException::new);
  boolean available = warehouseService.isAvailable(productId);
  int shipsInDays = available ? 0 : supplierService.getDeliveryTime(product.supplier(), productId);
  return new ProductPageResponse(product, shipsInDays, Thread.currentThread().toString());
}
```

Sequentieller Code ist nicht nur wesentlich einfacher zu schreiben und zu lesen, sondern auch leichter zu debuggen, da der Programmablauf in einem Debugger nachverfolgt werden kann und Stacktraces den erwarteten Aufrufstack widerspiegeln.

Ermöglicht wird das, indem virtuelle Threads – völlig transparent für die Entwicklerin oder den Entwickler – auf einige wenige Platform-Threads (so heißen die herkömmlichen Threads) gemappt werden. Sobald ein virtueller Thread warten muss oder blockiert wird, führt der Plattform-Thread einen anderen virtuellen Thread aus. So können wir mit wenigen Betriebssystem-Threads mehrere Millionen (!) virtuelle Threads ausführen.

Das Beste daran ist, dass wir den bestehenden Code nicht ändern müssen. Wir müssen lediglich unser Application Framework instruieren, keine Plattform-Threads, sondern virtuelle Threads zu erzeugen (z. B. durch *@RunOnVirtualThread*).

Wie genau virtuelle Threads funktionieren, was sie können und was nicht und für welche Einsatzzwecke sie nicht geeignet sind, kannst du unter https://www.happycoders.eu/de/java/virtual-threads/ nachlesen.

Sequenced Collections – JEP 431 🌟

Das neue *SequencedCollection*-Interface bietet einheitliche Methoden, um das erste und letzte Element einer geordneten *Collection*, wie z. B. *List*, *Deque*, *LinkedHashSet* oder *TreeSet*, zu lesen, zu entfernen oder einzufügen. Im folgenden findest du ein paar Beispiele, wie Java-Code durch die neuen Interface-Methoden stark vereinfacht werden kann:

Operation	Code vor Java 21	Code seit Java 21
Lesen des ersten Elements einer *List*	`list.get(0);`	`list.getFirst();`
Lesen des letzten Elements einer *List*	`list.get(list.size() - 1);`	`list.getLast();`
Lesen des ersten Elements eines *LinkedHashSet*	`linkedHashSet.iterator().next();`	`linkedHashSet.getFirst();`
Lesen des letzten Elements eines *LinkedHashSet*	*nur via Iteration über das komplette Set möglich*	`linkedHashSet.getLast();`

Weitere Methoden von *SequencedCollection* sind *addFirst(...)*, *addLast(...)*, *removeFirst(...)*, *removeLast(...)* und *reversed()*, womit über die *Collection* in umgekehrter Reihenfolge iteriert werden kann.

Ein weiteres Interface ist *SequencedMap*, was von *LinkedHashMap*, *TreeMap* und *ConcurrentSkipListMap* implementiert wird und die Methoden *firstEntry()*, *lastEntry()*, *pollFirstEntry()*, *pollLastEntry()*, *putFirst(...)*, *putLast(...)* und *reversed()* bereitstellt.

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 21 (Teil 2)

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.
Die nicht finalisierten JDK Enhancement Proposals aus Java 21 findest du jetzt auf den Seiten 2 und 3.

Pattern Matching for switch – JEP 441 🌟

Pattern Matching for switch ermöglicht es, Switch-Statements und -Ausdrücke über ein beliebiges Objekt zu formulieren. Als *case*-Labels sind nun auch Patterns, qualifizierte Enum-Konstanten (mit vorangestelltem Typnamen) und *null* erlaubt. Ein Beispiel:

```
Object o = . . .
switch (o) {
  case String s when s.length() > 5 -> System.out.println("o is a long String, its length is " + s.length());
  case String s                     -> System.out.println("o is a short String: " + s);
  case Month.JANUARY                -> System.out.println("o is the month of January");
  case null                         -> System.out.println("o is null");
  default                           -> System.out.println("o is something else");
}
```

Bei Verwendung eines der neuen Switch-Features (Pattern, qualifizierte Enum-Konstante oder *case null*-Label) muss der Switch erschöpfend sein, d. h. er muss alle erdenklichen Fälle abdecken oder ein *default*-Label enthalten.

Das ermöglicht – insbesondere in Kombination mit Sealed Classes – extrem ausdrucksstarken Code. Sei *Shape* eine versiegelte Klasse, die nur die Kindklassen *Rectangle* und *Circle* erlaubt. Dann gilt folgender Switch-Anweisung als vollständig:

```
Shape shape = . . .
switch (shape) {
  case Rectangle r -> System.out.printf("Rectangle: top left = %s; bottom right = %s%n", r.tl(), r.br());
  case Circle c    -> System.out.printf("Circle: center = %s; radius = %f%n", c.center(), c.radius());
}
```

Wenn du nun *Shape* um eine weitere Kindklasse erweiterst, wird der Compiler diesen Switch als nicht mehr vollständig bemängeln, und du wirst gezwungen sein, ihn um die neue Klasse zu erweitern. Das eliminiert viele potenzielle Fehlerquellen.

Record Patterns – JEP 440 🌟

Mit einem *Record Pattern* kann ein Java-Record beim Pattern Matching in seine Bestandteile dekonstruiert werden. Sei *Position* ein Record mit x und y-Koordinaten und *Path* ein Record mit einer Start- und einer End-Position. Dann ist z. B. folgendes möglich:

```
switch (o) {
  case Position(int x, int y)         -> System.out.printf("o is a position: %d/%d", x, y);
  case Path(Position(int x1, int y1),
            Position(int x2, int y2)) -> System.out.printf("o is a path: %d/%d -> %d/%d", x1, y1, x2, y2);
  case String s                       -> System.out.printf("o is a string: %s%n", s);
  default                             -> System.out.printf("o is something else: %s", o);
}
```

Generational ZGC – JEP 439 ⭐

Die „Weak Generational Hypothesis" besagt, dass die meisten Objekte kurz nach ihrer Erzeugung wieder sterben, und dass Objekte, die bereits einige GC-Zyklen überlebt haben, in der Regel noch länger am Leben bleiben.

Das macht sich ein sogenannter „Generational Garbage Collector" zunutze, indem er den Heap in zwei logische Bereiche aufteilt: eine „junge Generation", in der neue Objekte angelegt werden, und eine „alte Generation", in die Objekte verschoben werden, die ein gewisses Alter erreicht haben. Da Objekte in der alten Generation mit hoher Wahrscheinlichkeit *noch* älter werden, kann die Leistung einer Anwendung dadurch erhöht werden, dass der Garbage Collector die alte Generation seltener scannt.

Bisher machte der Z Gargabe Collector, kurz ZGC, keinen Unterschied zwischen „alten" und „neuen" Objekten. Das ändert sich in Java 21, wenn du den Generational ZGC mit folgender VM-Option aktivierst: *-XX:+UseZGC -XX:+ZGenerational*

Weitere Änderungen in Java 21

Generational Shenandoah (Experimental) (JEP 404) – Neben dem Z Garbage Collector soll auch der in Java 15 eingeführte „Shenandoah Garbage Collector" in Zukunft Generationen unterstützen.

Deprecate the Windows 32-bit x86 Port for Removal (JEP 449) – Die 32-Bit-Version von Windows 10 wird kaum noch verwendet, der Support endet im Oktober 2025, und Windows 11 – seit Oktober 2021 auf dem Markt – wurde nie in einer 32-Bit-Version angeboten. Entsprechend wurde die 32-Bit-Windows-Portierung als „deprecated for removal" markiert.

Prepare to Disallow the Dynamic Loading of Agents (JEP 451) – Das Laden von Agenten zur Laufzeit, wie z. B. durch einen Profiler, stellt ein nicht unerhebliches Sicherheitsrisiko dar und muss in Zukunft explizit erlaubt werden.

Key Encapsulation Mechanism API (JEP 452) – Key Encapsulation Mechanism (KEM) ist eine moderne Verschlüsselungstechnik, die den Austausch symmetrischer Schlüssel über ein asymmetrisches Verschlüsselungsverfahren ermöglicht.

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 20

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Die in Java 20 veröffentlichten Preview-Features – Scoped Values (JEP 429), Record Patterns (JEP 432), Pattern Matching for switch (JEP 433), Foreign Function & Memory API (JEP 434), Virtual Threads (JEP 436), Structured Concurrency (JEP 437) und Vector API (JEP 438) – wurden in Java 21 und Java 22 finalisiert bzw. befinden sich zum Stand von Java 22 noch im Preview-Stadium.

In Java 20 wurden keine finalen JDK Enhancement Proposals (JEPs) umgesetzt.
Neben den o. g. Preview-JEPs gab es folgende nennenswerte Änderungen:

Javac Warns about Type Casts in Compound Assignments with Possible Lossy Conversions ⭐

Fangen wir an mit einem kleinen Quiz. Was ist der Unterschied zwischen den folgenden zwei Statements?

```
a += b;
a = a + b;
```

Die meisten Java-Entwicklerinnen und -Entwickler werden sagen: Es gibt keinen. Doch das ist falsch. Wenn z. B. *a* ein *short* ist und *b* ein *int*, dann führt die zweite Zeile zu einem Compilerfehler:

```
java: incompatible types: possible lossy conversion from int to short
```

Denn *a* + *b* ergibt ein *int*, welches ohne expliziten Cast nicht der *short*-Variablen *a* zugewiesen werden darf.

Die erste Zeile hingegen ist erlaubt, da der Compiler bei einem sogenannten „Compound Assignment" einen *impliziten* Cast einfügt. Wenn *a* ein *short* ist, dann ist *a* += *b* äquivalent zu:

```
a = (short) (a + b);
```

Bei einem Cast von *int* auf *short* werden allerdings die oberen 16 Bit abgeschnitten. Es gehen also Informationen verloren, wie das folgende Beispiel zeigt:

```
short a = 30_000;
int b = 50_000;
a += b;
System.out.println("a = " + a);
```

Das Programm gibt nicht 80000 (hexadezimal 0x13880) aus, sondern 14464 (hexadezimal 0x3880).

Um Entwickler vor diesem potentiell unerwünschten Verhalten zu warnen, wurde in Java 20 (endlich!) eine entsprechende Compiler-Warnung eingeführt.

Idle Connection Timeouts for HTTP/2

Über die System Property *jdk.httpclient.keepalive.timeout* kann eingestellt werden, wie lange inaktive HTTP/1.1-Verbindungen offen gehalten werden. Ab Java 20 gilt diese Eigenschaft auch für HTTP/2-Verbindungen.

Desweiteren wurde die System Property *jdk.httpclient.keepalive.timeout.h2* hinzugefügt, mit der dieser Wert speziell für HTTP/2-Verbindungen überschrieben werden kann.

HttpClient Default Keep Alive Time is 30 Seconds

Wird die eben genannte System Property *jdk.httpclient.keepalive.timeout* nicht definiert, galt bis Java 19 ein Standardwert von 1.200 Sekunden. In Java 20 wurde der Standardwert auf 30 Sekunden herabgesetzt.

IdentityHashMap's Remove and Replace Methods Use Object Identity

IdentityHashMap ist eine spezielle *Map*-Implementierung, bei der Keys nicht dann als gleich gelten, wenn die *equals()*-Methode *true* ergibt, sondern dann, wenn die Key-Objekte *identisch* sind, also der Vergleich mittels ==-Operator *true* ergibt.

Als in Java 8 dem *Map*-Interface die Default-Methoden *remove(Object key, Object value)* und *replace(K key, V oldValue, V newValue)* hinzugefügt wurden, wurde allerdings vergessen, diese Methoden in *IdentityHashMap* so zu überschreiben, dass sie == anstelle von *equals()* verwenden.

Dieser Fehler wurde in Java 20, nach achteinhalb Jahren, korrigiert. Dass der Bug so lange nicht aufgefallen war, ist ein Zeichen dafür, dass *IdentityHashMap* wenig genutzt wird und möglicherweise weitere Bugs enthält.

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 19

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Die in Java 19 veröffentlichten Preview-Features – Record Patterns (JEP 405), Foreign Function & Memory API (JEP 424), Virtual Threads (JEP 425), Vector API (JEP 426), Pattern Matching for switch (JEP 427) und Structured Concurrency (JEP 428) – wurden in Java 21 und Java 22 finalisiert bzw. befinden sich zum Stand von Java 22 noch im Preview-Stadium.

New Methods to Create Preallocated HashMaps ⭐

HashMap bietet einen Konstruktor, mit dem man die initiale Kapazität der Map definieren kann, z. B. so:

```
Map<String, String> map = new HashMap<>(120);
```

Intuitiv würde man denken, diese *HashMap* biete Platz für 120 Einträge. Das ist allerdings nicht der Fall!

Denn die *HashMap* wird mit einem Default-Load-Faktor von 0,75 initialisiert. Das bedeutet: Sobald die HashMap zu 75 % gefüllt ist, wird sie mit doppelter Größe neu aufgebaut („rehashed"). Dadurch soll sichergestellt werden, dass die Elemente möglichst gleichmäßig auf die Buckets der *HashMap* verteilt sind und möglichst kein Bucket mehr als ein Element enthält.

Die mit einer Kapazität von 120 initialisierte *HashMap* kann also nur 120 × 0,75 = 90 Einträge aufnehmen.

Um eine *HashMap* für 120 Einträge zu erzeugen, musste man bisher die Kapazität selbst berechnen, indem man die Anzahl der Mappings durch den Load-Faktor teilt: 120 ÷ 0,75 = 160. Eine *HashMap* für 120 Einträge musste also wie folgt angelegt werden:

```
Map<String, Integer> map = new HashMap<>(160);
```

Java 19 macht es uns einfacher – wir können stattdessen jetzt folgendes schreiben:

```
Map<String, Integer> map = HashMap.newHashMap(120);
```

Die *newHashMap(...)*-Methode wurde ebenfalls in *LinkedHashMap* und *WeakHashMap* eingebaut. Schau gerne einmal in den Quellcode der neuen Methoden – du wirst sehen, dass sie im Endeffekt genau die Berechnung durch- führen, die wir zuvor manuell gemacht haben.

Additional Date-Time Formats ⭐

Über die Methoden *DateTimeFormatter.ofLocalizedDate(...)*, *ofLocalizedTime(...)* und *ofLocalizedDateTime(...)* und dem nach- folgenden Aufruf von *withLocale(...)* können wir uns seit Java 8 Datums- und Zeit-Formatierer erzeugen lassen. Das genaue Format steuern wir dabei über das Enum *FormatStyle*, das die Werte *FULL*, *LONG*, *MEDIUM* und *SHORT* annehmen kann.

In Java 19 kommt die Methode *ofLocalizedPattern(String requestedTemplate)* hinzu, mit der wir flexiblere Formate definieren können. Hier ein Beispiel:

```
DateTimeFormatter formatter = DateTimeFormatter.ofLocalizedPattern("yMMM");
System.out.println("US:      " + formatter.withLocale(Locale.US).format(LocalDate.now()));
System.out.println("Germany: " + formatter.withLocale(Locale.GERMANY).format(LocalDate.now()));
System.out.println("Japan:   " + formatter.withLocale(Locale.JAPAN).format(LocalDate.now()));
```

Der Code führt zu folgender Ausgabe:

```
US:      Jan 2024
Germany: Jan. 2024
Japan:   2024年1月
```

New System Properties for System.out and System.err

Falls die automatische Erkennung des konfigurierten Zeichensatzes der Konsole / des Terminals bei Ausgabe auf *System.out* oder *System.err* nicht funktionieren sollte, kannst du den Zeichensatz ab JDK 19 über folgende VM-Optionen einstellen, z. B. auf UTF-8:

```
-Dstdout.encoding=utf-8 -Dstderr.encoding=utf-8
```

Wenn du das nicht bei jedem Programmstart machen möchtest, kannst du diese Einstellungen auch global festlegen, indem du die folgende Umgebungsvariable definierst (ja, diese beginnt mit einem Unterstrich):

```
_JAVA_OPTIONS="-Dstdout.encoding=utf-8 -Dstderr.encoding=utf-8"
```

Linux/RISC-V Port – JEP 422

Aufgrund zunehmender Verbreitung von RISC-V-Hardware, wird ab Java 19 ein Linux-Port für die entsprechende Architektur zur Verfügung gestellt.

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 18

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Die in Java 18 veröffentlichten Preview-Features – Vector API (JEP 417), Foreign Function & Memory API (JEP 419) und Pattern Matching for switch (JEP 420) – wurden in Java 21 und Java 22 finalisiert bzw. befinden sich in Java 22 noch im Preview-Stadium.

UTF-8 by Default – JEP 400 🌟

Die Zeichencodierung bestimmt bei zahlreichen JDK-Methoden, wie Strings in Bytes umgewandelt werden und umgekehrt, z. B. durch *FileReader*, *FileWriter*, *InputStreamReader*, *InputStreamWriter*, *Formatter* und *Scanner*. Bis Java 17 hängt die Standard-Zeichencodierung von Betriebssystem und Sprache ab, was zu zahlreichen Problemen geführt hat, z. B. wenn eine Textdatei ohne explizite Angabe der Zeichencodierung auf einem Betriebssystem geschrieben und auf einem anderen gelesen wurde.

Die Standard-Zeichencodierung konnte mit der System Property *-Dfile.encoding* überschrieben werden, allerdings ignorieren neuere Methoden wie *Files.readString(...)* den so eingestellten Zeichensatz und verwenden grundsätzlich UTF-8.

Ab Java 18 ist die Standard-Zeichencodierung auf allen Plattformen und Sprachen einheitlich UTF-8, so dass die beschriebenen Probleme der Vergangenheit angehören sollten.

Code Snippets in Java API Documentation – JEP 413 🌟

Ab Java 18 können wir in JavaDoc-Kommentaren mit dem *@snippet*-Tag Code-Beispiele einfügen. Teile des Codes können dabei mit *@highight* markiert oder mit *@link* verlinkt werden, wie im Beispiel rechts.

Außerdem kannst du auf markierte Codestellen in anderen Java-Dateien verweisen. Mehr Details findest du im Java-18-Artikel.

```
/**
 * {@snippet :
 * // @highlight region regex="\bwrite.*?\b" type="highlighted":
 * // @link regex="\bBufferedWriter\b" target="java.io.BufferedWriter":
 * try (BufferedWriter writer = Files.newBufferedWriter(path)) {
 *  writer.write(text);
 * }
 * // @end
 * }
 */
```

Internet-Address Resolution SPI – JEP 418 ⭐

Um z. B. per *InetAddress.getByName(...)* die IP-Adresse zu einem Hostnamen zu finden, wird der Resolver des Betriebssystems aufgerufen, d. h. in der Regel wird zunächst die *hosts*-Datei konsultiert und danach die konfigurierten DNS-Server. Der Aufruf der entsprechenden Betriebssystem-Funktion ist blockierend und dauert mitunter recht lang. Außerdem können wir z. B. in Tests die Hostname-Auflösung nicht anpassen, um Requests auf einen Testserver umzuleiten.

Das Internet-Address Resolution SPI ermöglicht es, sehr einfach den Standard-Hostname-Resolver durch einen eigenen zu ersetzen. Wie das genau funktioniert, kannst du im Java-18-Artikel nachlesen.

Simple Web Server – JEP 408

Java 18 liefert einen rudimentären Webserver mit aus, den du mit dem Kommando rechts starten kannst.

```
$ jwebserver -b 127.0.0.100 -p 4444 -d /tmp
Serving /tmp and subdirectories on 127.0.0.100 port 4444
URL http://127.0.0.100:4444/
```

Durch *-b* gibst du die IP-Adresse an, auf der der Webserver Verbindungen annehmen soll, und durch *-p* den Port. Wenn du diese Angaben weglässt, hört der Webserver auf 127.0.0.1:8000. Der Parameter *-d* gibt das Verzeichnis an, das der Webserver ausliefern soll. Standardmäßig ist das das aktuelle Verzeichnis. Du kannst den Webserver auch aus Java-Code heraus starten (tatsächlich macht das *jwebserver*-Kommando genau das):

```
SimpleFileServer.createFileServer(
    new InetSocketAddress("127.0.0.10", 4444), Path.of("/tmp"), OutputLevel.INFO).start();
```

Reimplement Core Reflection with Method Handles – JEP 416

In Java gibt zwei Reflection-Mechanismen – *Core Reflection* und *Method* bzw. *Variable Handles*:

```
Field field = String.class.getDeclaredField("value");  // ← Core reflection
VarHandle handle = MethodHandles.privateLookupIn(String.class, MethodHandles.lookup())  // ← Method handles
                                .findVarHandle(String.class, "value", byte[].class);
```

Beide Mechanismen zu warten stellt einen erheblichen Mehraufwand dar. Daher wurde in Java 18 der Code der Reflection-Klassen *java.lang.reflect.Method*, *Field* und *Constructor* auf Basis von Method Handles neu implementiert.

Deprecate Finalization for Removal – JEP 421

Die seit Java 1.0 existierende *Finalization* ist unsicher, fehleranfällig und kann zu Performanceproblemen führen. Als Alternativen wurden in Java 7 *try-with-resources* und in Java 9 die *Cleaner API* eingeführt. Entsprechend wurden die *finalize()*-Methoden in *Object* und zahlreichen weiteren Klassen der JDK-Klassenbibliothek bereits in Java 9 als „deprecated" markiert. In Java 18 werden die *finalize()*-Methoden als „deprecated for removal" markiert.

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 17 (Teil 1)

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Sealed Classes – JEP 409 🌟

Bisher gab es nur begrenzte Möglichkeiten, um die Vererbbarkeit einer Klasse einzuschränken – durch *final* oder die Markierung als nicht-öffentlich. In Java 17 können wir mit den Keywords *sealed*, *non-sealed* und *permits* die Vererbarkeit wesentich feingranularer steuern. Im Beispiel rechts darf das Interface *Shape* nur von den Klassen *Circle* und *Square* implementiert werden.

```
public sealed class Shape permits Circle, Square {}
public final class Circle extends Shape {}
public final class Square extends Shape {}
```

Versiegelte Klassen ermöglichen z. B. die Erschöfungsanalyse bei dem in Java 21 eingeführten *Pattern Matching for switch*. Weitere Einsatzzwecke und Besonderheiten findest du unter https://www.happycoders.eu/de/java/sealed-classes/.

Strongly Encapsulate JDK Internals – JEP 403 🌟

Von Java 9 bis 15 hast Du vermutlich schon einmal die Warnung *„WARNING: An illegal reflective access operation has occurred"* zu sehen bekommen. Das liegt daran, dass für eine Übergangszeit nach der Einführung des Modulsystems Deep Reflection auf Code, der bereits vor Java 9 existierte, im sogenannten „Relaxed Strong Encapsulation"-Modus erlaubt war. In Java 16 wurde der Default-Modus auf „Strong Encapsulation" umgestellt, was anstatt zu einer Warnung dann zu einem Fehler und zum Programmabbruch führte. Mit der VM-Option *--illegal-access=permit* konnte in Java 16 das vorherige Verhalten wiederhergestellt werden.

In Java 17 gibt es nur noch den „Strong Encapsulation"-Modus; die VM-Option hat keinen Effekt mehr und erzeugt eine Warnung. Reflektiver Zugriff muss nun explizit für konkrete Module und Pakete mit folgender VM-Option erlaubt werden:

```
--add-opens <Name des aufgerufenen Moduls>/<Paketname>=<Name des aufrufenden Moduls oder ALL-UNNAMED>
```

Add java.time.InstantSource ⭐

Die Klasse *java.time.Clock* ist äußerst nützlich, um Tests zu schreiben, die zeitabhängige Funktionalität überprüfen. Wenn *Clock* z. B. per Dependency Injection in die Anwendungsklassen injiziert wird, kann man diese in Tests mocken und so z. B. durch *Clock.fixed(...)* eine spezifische Uhrzeit für den Test festlegen. Da *Clock* die Methode *getZone()* bereitstellt, muss man sich allerdings immer Gedanken darüber machen, mit welcher konkreten Zeitzone man ein *Clock*-Objekt instanziiert.

Um Zeitzonen-unabhängige Zeitquellen zu ermöglichen, wurde in Java 17 das Interface *java.time.InstantSource* aus *Clock* extrahiert, welches zum Abfragen der Zeit nur noch die Methoden *instant()* und *millis()* bereitstellt, wobei *millis()* bereits als Default-Methode implementiert ist.

Das Beispiel rechts oben zeigt eine Methode, die die Ausführungszeit eines *Runnable* misst. In Produktion können wir *Timer* mit der System-Uhr instanziieren:

```
Timer timer = new Timer(Clock.systemDefaultZone());
```

Testen können wir die Methode, indem wir *InstantSource* mocken, die *instant()*-Methode zwei feste Werte zurückgeben lassen und den Rückgabewert von *measure(...)* mit der Differenz dieser Werte vergleichen (s. Testmethode rechts).

```
public class Timer {
  private final InstantSource instantSource;

  public Timer(InstantSource instantSource) {
    this.instantSource = instantSource;
  }

  public Duration measure(Runnable runnable) {
    Instant start = instantSource.instant();
    runnable.run();
    Instant end = instantSource.instant();
    return Duration.between(start, end);
  }
}

@Test
void shouldReturnDurationBetweenStartAndEnd() {
  var instantSource = mock(InstantSource.class);

  when(instantSource.instant())
      .thenReturn(ofEpochMilli(1640033566000L))
      .thenReturn(ofEpochMilli(1640033566075L));

  Timer timer = new Timer(instantSource);
  Duration duration = timer.measure(() -> {});
  assertThat(duration).isEqualTo(ofMillis(75));
}
```

Hex Formatting and Parsing Utility ⭐

Java 17 stellt die neue Klasse *java.util.HexFormat* zur Verfügung, mit der die Darstellung und das Parsen hexadezimaler Zahlen über eine einheitliche API abgebildet wird. *HexFormat* unterstützt dabei alle primitiven Ganzzahlen (*int*, *byte*, *char*, *long*, *short*) sowie *byte*-Arrays – allerdings keine Fließkommazahlen. Hier ein Beispiel:

```
HexFormat hf = HexFormat.of();

System.out.println(hf.toHexDigits('A'));            // → 20ac
System.out.println(hf.toHexDigits((byte) 100));     // → 64
System.out.println(hf.toHexDigits((short) 10_000)); // → 2710
System.out.println(hf.toHexDigits(1_000_000_000));  // → 3b9aca00

System.out.println(hf.withUpperCase().formatHex(new byte[] {-54, -2, -70, -66})); // → CAFEBABE
System.out.println(hf.withDelimiter(" ").formatHex("Happy Coding!".getBytes()));  // → 48 61 70 70 79 ...
```

Weitere Methoden zur Darstellung und die Methoden zum Parsen von hexadezimalen Strings findest du im *HexFormat*-JavaDoc.

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 17 (Teil 2)

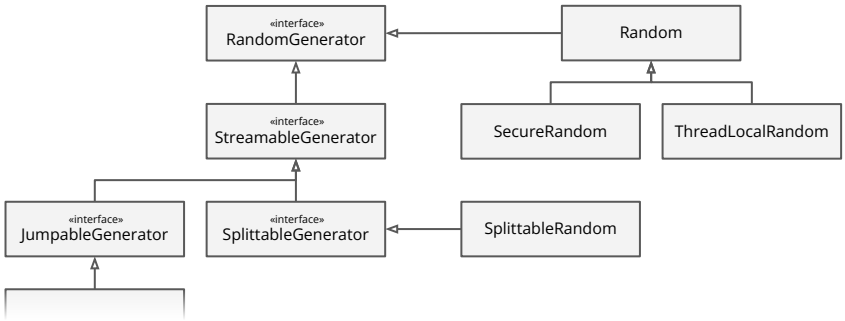
Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Die in Java 17 veröffentlichten Preview-Features – Pattern Matching for switch (JEP 406), Foreign Function & Memory API (JEP 412) und Vector API (JEP 414) – wurden in Java 21 und Java 22 finalisiert bzw. befinden sich in Java 22 noch im Preview-Stadium.

Enhanced Pseudo-Random Number Generators – JEP 356 ⭐

in Java 17 wurde ein Satz von voneinander erbenden Interfaces für bestehende und für neue Zufallszahlengeneratoren eingeführt, so dass durch die Verwendung von Interfaces konkrete Algorithmen deutlich einfacher als bisher austauschbar sind.

Die allen Zufallsgeneratoren gemeinsamen Methoden, wie *nextInt()* und *nextDouble()*, sind in *RandomGenerator* definiert. Sofern du nur diese Methoden benötigst, solltest du in Zukunft immer dieses Interface verwenden.



Context-Specific Deserialization Filters – JEP 415

Die Deserialisierung von Objekten stellt ein erhebliches Sicherheitsrisiko dar. Angreifer können über den zu deserialisierenden Datenstrom Objekte konstruieren, über die sie letztendlich beliebigen Code in beliebigen (auf dem Classpath verfügbaren) Klassen ausführen können. In Java 9 wurden globale und *ObjectInputStream*-spezifische Deserialisierungsfilter eingeführt, über die man festlegen kann, welche Klassen deserialisiert werden dürfen und welche nicht.

Java 17 erweitert diesen Schutzmechanismus um *kontextspezifische* Deserialisierungsfilter, die z. B. für einen bestimmten Thread, eine Klasse, ein Modul oder eine Drittanbieter-Bibliothek definiert werden können.

Unified Logging Supports Asynchronous Log Flushing

Bei asynchronem Logging werden Log-Nachrichten durch den Anwendungsthread zunächst in einen Puffer geschrieben. Ein separater I/O-Thread leitet sie dann an den konfigurierten Ausgang (Konsole, Datei oder Netzwerk) weiter. Der Anwendungsthread muss so nicht darauf warten, dass das entsprechende I/O-Subsytem die Nachricht verarbeitet hat.

Während die meisten Logging-Frameworks asynchrones Logging unterstützen, konnten Log-Nachrichten der JVM selbst (z. B. die des Garbage Collectors) bisher ausschließlich synchron weggeschrieben werden.

Ab Java 17 kann über die VM-Option *-Xlog:async* auch für die JVM selbst asynchrones Logging aktiviert werden. Die Größe des Log-Puffers kann über *-XX:AsyncLogBufferSize=<Bytes>* festgelegt werden.

Weitere Änderungen in Java 17

Restore Always-Strict Floating-Point Semantics (JEP 306) – Vor Java 1.2 wurden Fließkommaoperationen standardmäßig „strikt" ausgeführt, d. h. sie führten auf allen Architekturen zum identischen Ergebnis. Ab Java 1.2 wurde auf die damals noch schnellere „Standard-Fließkomma-Semantik" der Prozessorarchitektur gewechselt. Wer weiter strikt rechnen wollte, musste eine Klasse mit dem Keyword *strictfp* markieren. Da moderne CPUs strikte Operationen ohne Performance-Einbußen durchführen, wird ab Java 17 wieder standardmäßig strikt gerechnet. Das *strictfp*-Keyword ist nun überflüssig und führt nun zu einer Compiler-Warnung.

New macOS Rendering Pipeline (JEP 382) – Die Swing-Rendering-Pipeline für macOS wird auf das Metal-Framework umgestellt, das 2018 die zuvor unter macOS genutzte OpenGL-Bibliothek ersetzt hat.

macOS/AArch64 Port (JEP 391) – Apple hat angekündigt, Macs langfristig von x64- auf AArch64-CPUs umzustellen. Dementsprechend wird ein AArch64-Port bereitgestellt.

Deprecate the Applet API for Removal (JEP 398) – Java Applets werden von keinem modernen Webbrowser mehr unterstützt und wurden bereits in Java 9 als „deprecated" markiert. In Java 17 werden sie als „deprecated for removal" markiert.

Remove RMI Activation (JEP 407) – Durch RMI Activation können Objekte, die auf der Ziel-JVM zerstört wurden, wieder instanziiert werden, sobald auf sie zugegriffen wird. Da dieses Feature aufwändig zu warten und kaum genutzt war, wurde es entfernt.

Remove the Experimental AOT and JIT Compiler (JEP 410) – Der in Java 9 und 10 als AOT- bzw. JIT-Compiler bereitgestellte Graal Compiler wurde nur wenig genutzt und in Java 16 aus dem Oracle JDK und in Java 17 auch aus dem OpenJDK wieder entfernt.

Deprecate the Security Manager for Removal (JEP 411) – Der Security Manager sollte den Rechner vor heruntergeladenen Java-Applets schützen. Genau wie Java-Applets wurde nun auch der Security Manager als „deprecated for removal" markiert.

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 16

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Die in Java 16 veröffentlichten Previews – Vector API (JEP 338), Foreign Linker API (JEP 389), Foreign-Memory Access API (JEP 393) und Sealed Classes (JEP 397) – werden in Java 17 und Java 22 finalisiert bzw. befinden sich in Java 22 noch im Preview-Stadium.

Pattern Matching for instanceof – JEP 394 🌟

Ab Java 16 müssen wir eine Variable nach einer *instanceof*-Prüfung nicht mehr explizit auf den Zieltyp casten. Stattdessen können wir die Variable an eine neue Variable des entsprechenden Typs binden, z. B. so:

```
if (o instanceof String s && !s.isEmpty()) System.out.println("o in Großbuchstaben: " + s.toUpperCase());
```

Records – JEP 395 🌟

Java-Records bieten eine kompakte Notationsmöglichkeit, um Klassen mit ausschließlich finalen Feldern zu definieren.

Welche Besonderheiten du bei der Arbeit mit Records beachten musst, erfährst du in https://www.happycoders.eu/de/java/java-records/.

```
record Position(int x, int y) {}
record Path(Point start, Point end) {}

Path p = new Path(new Pos(0,8), new Pos(1,5));
int startX = p.start().x();
```

Strongly Encapsulate JDK Internals by Default – JEP 396 🌟

Du hast vermutlich schon einmal die Warnung *„WARNING: An illegal reflective access operation has occurred"* zu sehen bekommen. Das liegt daran, dass für eine Übergangszeit nach der Einführung des Java-Modulsystems in JDK 9 Deep Reflection auf Code, der bereits vor Java 9 existierte, standardmäßig im sogenannten „Relaxed Strong Encapsulation"-Modus erlaubt war.

In Java 16 wird der Default-Modus auf „Strong Encapsulation" umgestellt, was anstatt zu einer Warnung nun zu einem Fehler und zum Programmabbruch führt. Mit der VM-Option *--illegal-access=permit* kannst du das vorherige Verhalten vorübergehend wieder- herstellen. Die nachhaltigere Lösung ist es allerdings, reflektiven Zugriff explizit mit folgender VM-Option zu erlauben:

```
--add-opens <Name des aufgerufenen Moduls>/<Paketname>=<Name des aufrufenden Moduls oder ALL-UNNAMED>
```

Neue Stream-Methoden 🌟

Die neue *Stream.toList()*-Methode ist eine Kurzform für das häufig benötigte *Stream.collect(Collectors.toUnmodifiableList())*.

Stream.mapMulti(...) ist eine Alternative zu *flatMap(...)*, die ohne Erzeugung temporärer Streams auskommt. Wie genau *mapMulti(...)* funktioniert kannst du im Artikel über Java 16 nachlesen.

Warnings for Value-Based Classes – JEP 390 ⭐

Folgende Klassen wurden mit der Annotation *@ValueBased* versehen, ihre Konstruktoren als „deprecated for removal markiert":

- alle Wrapper-Klassen der primitiven Datentypen (*Byte*, *Short*, *Integer*, *Long*, *Float*, *Double*, *Boolean* und *Character*),
- *Optional* und dessen primitive Varianten,
- Zahlreiche Klassen der Date/Time-API, wie z. B. *LocalDateTime*,
- die durch *List.of()*, *Set.of()* und *Map.of()* erzeugten Collections.

Dies ist eine Vorbereitung auf die in Project Valhalla entwickelten *Value Objects*, also Objekte ohne Identität und Header.

Migrate from Mercurial to Git + Migrate to GitHub – JEPs 357 + 369 ⭐

Mit Java 16 wurde der JDK-Code vom Versionsverwaltungssystem Mercurial auf das weitaus verbreitetere Git migriert. Als Hosting-Platform wurde GitHub gewählt. Der JDK-Quellcode steht seither hier zur Verfügung: https://github.com/openjdk/

Weitere Änderungen in Java 16

Enable C++14 Language Features (JEP 347) – Der C++-Anteil des JDK war bisher auf die Sprachspezifikation C++98/03 – ein über 20 Jahre alter Standard – beschränkt. Im JDK 16 wird die Unterstützung auf C++14 angehoben.

ZGC: Concurrent Thread-Stack Processing (JEP 376) – Das sogenannte „Thread-Stack Processing", das im ZGC bisher noch in sogenannten Safepoints, also in Stop-the-World-Phasen, ausgeführt wurde, läuft ab Java 16 parallel zur laufenden Anwendung.

Unix-Domain Socket Channels (JEP 380) – Java unterstützt ab Version 16 sogenannte Unix-Domain-Sockets, die für die Inter-Prozess-Kommunikation (IPC) innerhalb eines Hosts verwendet werden und deutlich schneller sind als TCP-Sockets.

Alpine Linux Port (JEP 386) – Das JDK wird um einen Port für das im Cloud-Umfeld äußerst beliebte Alpine-Linux erweitert.

Elastic Metaspace (JEP 387) – In Java 16 wurde der Memory-Footprint des Metaspace, also des Speichers, in dem Klassen-Metadaten abgelegt werden, gesenkt. Metaspace-Speicher wird außerdem schneller an das Betriebssystem zurückgegeben.

Windows/AArch64 Port (JEP 388) – Windows auf der ARM64/AArch64-Prozessorarchitektur ist zu einer wichtigen Plattform geworden. Entsprechend wird auch dafür ein Port, basierend auf dem bereits existierenden Linux/AArch64-Port, bereitgestellt.

Packaging Tool (JEP 392) – Das Paketierungstool *jpackage* verpackt eine Java-Anwendung mitsamt der Java-Laufzeitumgebung (also der JVM und den benötigten Modulen der Klassenbibliothek) in ein Installationspaket für verschiedene Betriebssysteme.

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 15

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Die in Java 15 veröffentlichten Preview-Features – Sealed Classes (JEP 360), Pattern Matching for instanceof (JEP 375), Foreign-Memory Access API (JEP 383) und Records (JEP 384) – werden in Java 16, 17 und 22 finalisiert.

Text Blocks – JEP 378 🌟

Ab Java 15 kannst du mehrzeilige Strings, wie rechts gezeigt, durch je drei Anführungszeichen begrenzt, codieren. Einzelne Anführungszeichen brauchst du nicht mehr zu escapen. Die allen Zeilen gemeinsame Einrückung wird entfernt.

Details findest du in https://www.happycoders.eu/de/java/java-text-blocks/.

```
String sql = """
    SELECT id, title, text FROM Article
     WHERE category = "Java"
  ORDER BY title""";
```

Neue String- und CharSequence-Methoden 🌟

Die Klassen *String* und *CharSequence* wurden in Java 15 um folgende Methoden erweitert:

- *String.formatted(Object... args)* – Platzhalter in Strings können nun auch beispielsweise wie folgt ersetzt werden:
  `String message = "User %,d %s logged in at %s".formatted(userId, username, ZonedDateTime.now())`
- *String.stripIndent()* – entfernt die gleiche Anzahl Leerzeichen vom Anfang jeder Zeile des Strings, so dass die Zeile mit der zuvor geringsten Anzahl an Leerzeichen am Zeilenanfang beginnt. Entfernt zudem alle abschließenden Leerzeichen.
- *String.translateEscapes()* – ersetzt escapete Escape-Sequenzen, wie „\\n" durch ausgewertete Escape-Sequenzen wie „\n".
- *CharSequence.isEmpty()* – prüft, ob eine Zeichenfolge, z. B. ein *String* oder *StringBuilder*, leer ist.

ZGC: A Scalable Low-Latency Garbage Collector – JEP 377 ⭐

Der Z Garbage Collector, kurz ZGC, verspricht Pausezeiten von 10 ms nicht zu überschreiten und dabei den Gesamtdurchsatz der Anwendung um nicht mehr als 15 % gegenüber dem G1 Garbage Collector (G1GC) zu reduzieren (die Verringerung des Durchsatzes sind die Kosten für die niedrige Latenz). ZGC unterstützt Heap-Größen bis zu 16 TB. Genau wie G1 basiert ZGC auf Regionen, ist NUMA-aware und gibt ungenutzten Speicher an das Betriebssystem zurück.

Aktiviert wird ZGC mit der folgenden VM-Option: *-XX:+UseZGC*

Mit der VM-Option *-XX:SoftMaxHeapSize* lässt sich zudem eine „weiche" Heap-Obergrenze konfigurieren, die ZGC nur dann überschreitet, wenn es zur Vermeidung eines *OutOfMemoryErrors* zwingend notwendig ist.

Shenandoah: A Low-Pause-Time Garbage Collector – JEP 379

Auch Shenandoah verspricht minimale Pausezeiten unabhängig von der Heap-Größe. Shenandoah gibt ebenfalls ungenutzten Speicherplatz ans Betriebssystem zurück. Support für NUMA und *SoftMaxHeapSize* gibt es aktuell allerdings nicht.

Aktivieren kannst du Shenandoah mit der folgenden VM-Option: *-XX:+UseShenandoahGC*

Weitere Änderungen in Java 15

Edwards-Curve Digital Signature Algorithm (EdDSA) (JEP 339) – EdDSA ist ein modernes Signaturverfahren, das bei gleicher Sicherheitsstärke schneller ist als bisherige Signaturverfahren wie DSA und ECDSA.

Hidden Classes (JEP 371) – Frameworks wie Jakarta EE und Spring generieren zahlreiche Klassen dynamisch zur Laufzeit. Diese sind von regulären Klassen bisher nicht zu unterscheiden und somit für alle anderen Klassen sichtbar. Diese Framework-spezifischen Klassen können ab Java 15 als „versteckt" definiert werden und bleiben so vor dem Rest der Anwendung verborgen.

Remove the Nashorn JavaScript Engine (JEP 372) – Die in Java 8 eingeführte JavaScript-Engine „Nashorn" wurde in Java 11 als „deprecated for removal" markiert und wird in Java 15 vollständig enfernt.

Reimplement the Legacy DatagramSocket API (JEP 373) – Die seit Java 1.0 existierenden APIs *java.net.DatagramSocket* und *java.net.MulticastSocket* bestehen aus einer Mischung aus veraltetem, schwer wart- und erweiterbarem Java- und C-Code. Der Code enthält einige Bugs und lässt sich nicht ohne weiteres an die in Projekt Loom entwickelten virtuellen Threads anpassen. Daher wird die alte Implementierung durch eine einfachere, modernere, leichter wartbare Implementierung ersetzt.

Disable and Deprecate Biased Locking (JEP 374) – Biased Locking ist eine Optimierung der Thread-Synchronisierung, von der hauptsächlich Klassen wie *Vector*, *Hashtable* und *StringBuffer* profitieren, bei denen jeder Zugriff synchronisiert ist. Da diese Klassen kaum noch genutzt werden und Biased Locking schwer zu warten ist, wird es deaktiviert und als „deprecated" markiert.

Remove the Solaris and SPARC Ports (JEP 381) – Die nicht mehr zeitgemäßen Solaris- und SPARC-Ports wurden in Java 14 als „deprecated" markiert und werden in Java 15 aus dem JDK entfernt.

Deprecate RMI Activation for Removal (JEP 385) – Durch RMI (Remote Method Invocation) Activation können Objekte, die auf der Ziel-JVM zerstört wurden, automatisch wieder instanziiert werden, sobald auf sie zugegriffen wird. Da dieses Feature aufwändig zu warten und kaum genutzt ist, wird es als „deprecated" markiert.

Komprimierte Heap Dumps – Bei dem *jcmd*-Kommando *GC.heap_dump* kann ab Java 15 mit der Option *-gz=<Kompressionslevel>* der Heap-Dump ZIP-komprimiert gespeichert werden. Level 1 ist die schnellste, Level 9 die stärkste Kompression.

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 14

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Die in Java 14 veröffentlichten Preview-Features – Pattern Matching for instanceof (JEP 305), Packaging Tool (JEP 343), Records (JEP 359), ZGC on macOS / Windows (JEPs 364 + 365), Text Blocks (JEP 368) und Foreign-Memory Access API (JEP 370) – werden in Java 15, 16 und 22 finalisiert.

Switch Expressions – JEP 361 🌟

Switch-Anweisungen können ab Java 14 in der sogenannten Pfeil-Notation geschrieben werden. Auf eine oder mehrere Konstanten folgt ein Pfeil und darauf eine einzelne Anweisung oder ein Codeblock. Das fehleranfällige, von C geerbte Fallthrough zum nächsten Label gibt es bei dieser Notation nicht mehr (s. Beispiel 1).

Switch kann darüber hinaus nun auch als Ausdruck (Expression) eingesetzt werden und einen Wert zurückgeben. Mit einem Switch-Ausdruck können wir z. B. recht elegant eine rekursive Fibonacci-Funktion schreiben (s. Beispiel 2).

Weitere Details zu Switch Expressions erfährst du in: www.happycoders.eu/de/java/switch-expressions/

Beispiel 1:

```
switch (dayOfWeek) {
  case MONDAY, TUESDAY     -> System.out.print("Beginning");
  case WEDNESDAY, THURSDAY -> System.out.print("Middle");
  case FRIDAY              -> System.out.print("Almost!!!");
  case SATURDAY, SUNDAY    -> System.out.print("Weekend");
}
```

Beispiel 2:

```
public static int fibonacci(int i) {
  return switch (i) {
    case 0, 1 -> i;
    default   -> fibonacci(i - 1) + fibonacci(i - 2);
  };
}
```

Helpful NullPointerExceptions – JEP 358 🌟

Eine *NullPointerException* zeigt dir ab Java 14 nicht mehr nur, in welcher Zeile sie aufgetreten ist, sondern auch, welches Objekt in dieser Zeile *null* ist, z. B. so:

```
Exception in thread "main" java.lang.NullPointerException:
Cannot invoke "Map.getValue()" because the return value of "Container.getMap()" is null
    at eu.happycoders.BusinessLogic.calculate(BusinessLogic.java:80)
```

Accounting Currency Format Support

In manchen Ländern, z. B. in den USA, werden negative Zahlen in der Buchhaltung nicht mit einem Minuszeichen, sondern durch runde Klammern gekennzeichnet. In Java 14 wird die sogenannte Language-Tag Extension „u-cf-account" hinzugefügt, die es ermöglicht, in einem *Locale*-Objekt die Zusatzinformation anzugeben, ob dieses im Kontext der Buchhaltung verwendet wird:

```
NumberFormat nf           = NumberFormat.getCurrencyInstance(Locale.US);
NumberFormat nfAccounting = NumberFormat.getCurrencyInstance(Locale.forLanguageTag("en-US-u-cf-account"));
System.out.println("Normal:     " + nf           .format(-14.95));  // → -$14.95
System.out.println("Accounting: " + nfAccounting.format(-14.95));  // → ($14.95)
```

Weitere Änderungen in Java 14

Non-Volatile Mapped Byte Buffers (JEP 352) – Mit einem *MappedByteBuffer* kannst du eine Datei in den Speicher mappen, um sie dann über reguläre Speicherzugriffe lesen und ändern zu können. Geänderte Daten müssen regelmäßig ins Speichermedium übertragen werden. Für NVMs gibt es hierfür effizientere Methoden als bei herkömmlichen Speichermedien. Ab Java 14 kannst du dafür beim Aufruf von *FileChannel.map(...)* die Modi *ExtendedMapMode.READ_ONLY_SYNC* oder *READ_WRITE_SYNC* angeben.

NUMA-Aware Memory Allocation for G1 (JEP 345) – Auf modernen Servern spielt die physische Distanz zwischen CPU-Kern und Speichermodul eine immer größere Rolle. Der G1GC kann NUMA (Non-Uniform Memory Access)-Daten nutzen, um Objekte auf demjenigen Speicherknoten zu allokieren, der dem CPU-Kern am nächsten ist, auf dem der Objekt-erzeugende Thread läuft. *NUMA-Aware Memory Allocation* ist nur für Linux verfügbar und muss über die VM-Option *+XX:+UseNUMA* aktiviert werden.

JFR Event Streaming (JEP 349) – Der Java Flight Recorder (JFR) sammelt während der Ausführung einer Anwendung Daten über die JVM und speichert sie in einer Datei. Die Daten können dann mit JDK Mission Control visualisiert werden. Ab Java 14 ist auch eine kontinuierliche Überwachung möglich. Wie das funktioniert, erfährst du im Java-14-Artikel.

Deprecate the Solaris and SPARC Ports (JEP 362) – Solaris und die SPARC-Prozessorarchitektur sind nicht mehr zeitgemäß. Um Entwicklungsresourcen anderweitig einsetzen zu können, werden die entsprechenden Ports als „deprecated" markiert.

Remove the Concurrent Mark Sweep (CMS) Garbage Collector (JEP 363) – Der Concurrent Mark Sweep (CMS) Garbage Collector wurde in Java 9 als „deprecated" markiert. Die Entwicklungsressourcen sollen zugunsten modernerer Garbage Collectoren wie G1GC (Standard seit Java 9) und ZGC umverteilt werden. CMS wird in Java 14 vollständig aus dem JDK entfernt.

Deprecate the ParallelScavenge + SerialOld GC Combination (JEP 366) – Die selten genutzte Kombination von GC-Algorithmen, parallel für die junge Generation und seriell für die alten Generation, aktiviert durch *-XX:+UseParallelGC -XX:-UseParallelOldGC*, wird selten genutzt und erfordert einen hohen Wartungsaufwand. Sie wird daher als „deprecated" markiert.

Remove the Pack200 Tools and API (JEP 367) – Das Kompressionsverfahren „Pack200" wurde in Java 5 eingeführt, um in den frühen 2000ern durch effektivere Kompression möglichst viel Bandbreite einzusparen. Die Entwicklungskosten stehen in Zeiten von 100-MBit-Leitungen in keinem Verhältnis zum Nutzen mehr. Pack200 wird daher in Java 14 entfernt.

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 13

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Die in Java 13 veröffentlichten Preview-Features – Switch Expressions (JEP 354), Text Blocks (JEP 355) und ZGC: Uncommit Unused Memory (351) – werden allesamt in Java 15 finalisiert.

Dynamic CDS Archives – JEP 350

Application Class-Data Sharing (s. Java 10) was bisher aufwändig zu konfigurieren. Außerdem wurden nicht nur die Klassen der Anwendung selbst, sondern auch alle genutzten Klassen des JDK in das Shared Archive mit aufgenommen.

In Java 13 werden nur noch die Klassen der Anwendung im Shared Archive gespeichert; die JDK-Klassen werden aus dem seit Java 12 mitgelieferten Basis-Archiv *classes.jsa* geladen. Das Shared Archive eines simplen Hello-World-Programms wird dadurch von 9 MB auf 256 KB verkleinert.

Ein Shared Archive für eine nach *target/helloworld.jar* kompilierte Hello-World-Anwendung mit einer *main*-Methode in der Klasse *eu.happycoders.helloworld.Main* erzeugst du beispielsweise wie folgt:

```
java -XX:ArchiveClassesAtExit=helloworld.jsa -classpath target/helloworld.jar eu.happycoders.helloworld.Main
```

Um das Programm unter Verwendung des Shared Archives zu starten, rufst du es danach wie folgt auf:

```
java -XX:SharedArchiveFile=helloworld.jsa -classpath target/helloworld.jar eu.happycoders.helloworld.Main
```

Reimplement the Legacy Socket API – JEP 353

Die APIs *java.net.Socket* und *java.net.ServerSocket* existieren seit Java 1.0. Der zugrunde liegende Code, eine Mischung aus Java- und C-Code, ist schwer wart- und erweiterbar – insbesondere im Hinblick auf Project Loom, mit dem in Java 21 virtuelle Threads eingeführt werden.

In Java 13 wird die alte Implementierung durch eine modernere, besser wart- und erweiterbare Implementierung ersetzt, die sich inbesondere ohne weitere Refactorings an virtuelle Threads anpassen lassen soll.

Buffer.slice()

Mit der neuen Methode *Buffer.slice(int index, int length)* kannst du eine View auf einen Teilbereich eines Puffers (z. B. *ByteBuffer*, *IntBuffer*, *DoubleBuffer*) erzeugen, die an Position *index* beginnt und *length* Bytes enthält.

Neue XxxBuffer.get()- und put()-Methoden

Analog gibt es jeweils zwei neue *get()*- und *put()*-Methoden in den konkreten *Buffer*-Klassen *ByteBuffer*, *IntBuffer*, *DoubleBuffer*, usw., mit denen die Daten nicht wie bisher an der aktuellen Position des Puffers gelesen/geschrieben werden, sondern an der explizit angegeben Position. Im folgenden siehst du beispielhaft die neuen Methoden der konkreten Klasse *ByteBuffer*:

- *get(int index, byte[] dst, int offset, int length)*
  überträgt *length* Bytes ab der durch *index* angegebenen Position dieses Puffers in das Byte-Array *dst* ab Position *offset*.
- *get(int index, byte[] dst)*
  überträgt Daten ab der durch *index* angegebenen Position dieses Puffers in das Byte-Array *dst*. Die Anzahl der übertragenen Bytes entspricht der Länge des Ziel-Arrays.
- *put(int index, byte[] src, int offset, int length)*
  überträgt *length* Bytes aus dem Byte-Array *src* ab Position *offset* in diesen Puffer ab Position *index*.
- *put(int index, byte[] src)*
  überträgt alle Bytes aus dem Byte-Array *src* in diesen Puffer ab Position *index*.

Die Position des Puffers bleibt bei allen vier Methoden unverändert.

FileSystems.newFileSystem()

Mit der Methode *FileSystems.newFileSystem(Path path, ClassLoader loader)* kannst du ein Pseudo-Dateisystem erstellen, dessen Inhalt auf eine Datei (wie z. B. eine ZIP- oder JAR-Datei) gemappt wird.

Die Methode wurde in Java 13 mit einer Variante überladen, die es ermöglicht, über den Parameter *env* eine anbieterspezifische Konfiguration des Dateisystems zu übergeben: *FileSystems.newFileSystem(Path path, Map env, ClassLoader loader)*.

Weiterhin wurden zwei Varianten jeweils ohne den *loader*-Parameter hinzugefügt. Ein Class Loader wird nur dann benötigt, wenn der sogenannte *FileSystemProvider* für den zu mappenden Dateityp nicht im JDK registriert ist, sondern über den angegebenen Class Loader geladen werden soll. Für gängige Dateitypen wie ZIP oder JAR ist das nicht erforderlich.

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 12

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Die in Java 12 veröffentlichten Preview-Features – Switch Expressions (JEP 325) und Shenandoah: A Low-Pause-Time Garbage Collector (JEP 189) – werden in Java 14 und Java 15 finalisiert.

Neue String- und Files-Methoden 🌟

In Java 12 wurden die folgenden Methoden zu den *String*- und *Files*-Klassen hinzugefügt:

- *String.indent(int n)* – rückt den String, auf dem *indent(...)* aufgerufen wird, zeilenweise um *n* Leerzeichen ein.
- *String.transform(Function<...> f)* – ruft die Funktion *f* mit diesem String als Parameter auf und gibt das Ergebnis zurück.
- *Files.mismatch()* – gibt die Position des ersten Bytes an, in dem sich die Dateien unterscheiden, oder -1 bei gleichen Dateien.

Teeing-Collector ⭐

Der „Teeing Collector" ermöglicht es, in einem Stream zwei Collectoren anzugeben und eine Merger-Funktion, die die Ergebnisse der Collectoren kombiniert. Das folgende Beispiel ermittelt aus einem Stream die Differenz zwischen Maximum und Minimum:

```
long width = numbersStream.collect(
        Collectors.teeing(Collectors.minBy(Integer::compareTo), Collectors.maxBy(Integer::compareTo),
                          (min, max) -> (long) max.orElseThrow() - min.orElseThrow()));
```

Support for Compact Number Formatting ⭐

Mit *NumberFormat.getCompactNumberInstance(...)* können wir einen Formatierer für die sogenannte „kompakte Zahlenformatierung" erzeugen. Das ist eine für Menschen leicht lesbare Form, wie z. B. „2 Mio." oder „3 Milliarden". Hier ein Beispiel:

```
NumberFormat nfShort = NumberFormat.getCompactNumberInstance(Locale.GERMANY, NumberFormat.Style.SHORT);
NumberFormat nfLong  = NumberFormat.getCompactNumberInstance(Locale.GERMANY, NumberFormat.Style.LONG);
System.out.println(nfShort.format(1_000));          // → 1.000
System.out.println(nfShort.format(3_456_789_000L)); // → 3 Mrd.
System.out.println(nfLong.format(1_000));           // → 1 Tausend
System.out.println(nfLong.format(3_456_789_000L));  // → 3 Milliarden
```

Default CDS Archives – JEP 341

Um *Class-Data Sharing* (s. Java 11) zu aktivieren, musstest du bisher für jede Java-Installation einmalig das Kommando *java -Xshare:dump* ausführen, um die Shared-Archive-Datei *classes.jsa* zu generieren.

Ab Java 12 wird das JDK auf 64-Bit-Architekturen mitsamt dieser Datei ausgeliefert, so dass die Ausführung des oben genannten Kommandos nicht mehr nötig ist und Java-Anwendungen standardmäßig das Default-CDS-Archiv verwenden.

Abortable Mixed Collections for G1 – JEP 344

Eines der Ziele des G1 Gargabe Collectors ist es, für „Stop-the-World"-Phasen vorgegebene maximale Pausezeiten einzuhalten – also die Anwendung nicht länger als die vorgegebene Zeit zu stoppen. Dafür verwendet G1 eine Heuristik, die insbesondere bei geändertem Verhalten der Anwendung dazu führen kann, dass die Anwendung länger als vorgesehen pausiert wird.

In Java 12 wurden „Mixed Collections" (d. h. Aufräumarbeiten, die sowohl die junge als auch die alte Generation betreffen) in einen obligatorischen und einen optionalen Teil aufgeteilt. Der obligatorische Teil wird ununterbrechbar ausgeführt – und der optionale Teil in inkrementellen Schritten, bis die maximale Pausezeit erreicht ist. Währenddessen versucht G1, die Heuristik so anzupassen, dass sie wieder Collection Sets bestimmt, die in der maximalen Pausezeit komplett abarbeitbar sind.

Promptly Return Unused Committed Memory from G1 – JEP 346

In Cloud-Umgebungen, in denen man für den tatsächlich genutzten Arbeitsspeicher bezahlt, ist es wünschenswert, dass der Garbage Collector ungenutzten Speicherplatz schnell wieder an das Betriebssystem zurückgibt. Der G1 Garbage Collector kann zwar Speicherplatz zurückgeben, allerdings nur während der Garbage-Collection-Phase. Wenn jedoch die Heap-Belegung oder die aktuelle Rate der Objekt-Allokation keinen GC-Zyklus triggert, wird auch kein Arbeitsspeicher zurückgegeben.

Ab Java 12 kannst du mit der VM-Option *-XX:G1PeriodicGCInterval=<milliseconds>* ein Intervall angeben, in dem GC-Zyklen durchgeführt werden sollen, unabhängig davon, ob sie nötig sind oder nicht.

Weitere Änderungen in Java 12

Microbenchmark Suite (JEP 230) – Microbenchmarks, die regelmäßig die Leistung der JDK-Klassenbibliothek messen, wurden bisher als separates Projekt verwaltet. In Java 12 wurde der Code dieser Benchmarks in den JDK-Quellcode verschoben.

JVM Constants API (JEP 334) – Im Konstantenpool einer *.class*-Datei werden u. a. Strings und die Namen referenzierter Klassen und Methoden gespeichert. Die JVM Constants API soll es erleichtern, Java-Programme zu schreiben, die JVM-Bytecode manipulieren. Dazu stellt die API neue Interfaces und Klassen bereit, mit der der Constant Pool verwaltet werden kann.

One AArch64 Port, Not Two (JEP 340) – Im JDK existieren zwei Portierungen für 64-Bit ARM-CPUs: „arm64" von Oracle und „aarch64" von Red Hat. In Java 12 wird der Port von Oracle entfernt, um Entwicklungsresourcen zu konsolidieren.

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 11 (Teil 1)

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Launch Single-File Source-Code Programs – JEP 330 🌟

Eine der nützlichsten Erweiterungen in Java 11 ist, dass Java-Programme, die aus nur einer Datei bestehen, ohne explizites Kompilieren gestartet werden können. Speichere das Programm rechts in der Datei *Hello.java* ab. Du kannst es dann wie folgt ausführen, ohne es vorab mit *javac* kompilieren zu müssen:

```
$ java Hello.java
```

```
public class Hello {
  public static void main(String[] args) {
    System.out.println("Hello, world!");
  }
}
```

Local-Variable Syntax for Lambda Parameters – JEP 323 🌟

Ein gern genutztes Beispiel für die Verwendung eines Lambdaausdrucks ist:

```
map.computeIfAbsent(key, (String k) -> new ArrayList<>()).add(value);
```

Java kann den Typ des Lambda-Parameters *k* aus dem Kontext herleiten, daher kannst du ihn auch weglassen:

```
map.computeIfAbsent(key, k -> new ArrayList<>()).add(value);
```

Ab Java 11 darfst du statt eines konkreten Typs auch *var* angeben:

```
map.computeIfAbsent(key, (var k) -> new ArrayList<>()).add(value);
```

Welchen Sinn macht es *var* anzugeben, wenn es auch ohne geht? Der Grund dafür ist, dass nur konkrete Typen sowie *var* mit Annotationen versehen werden dürfen, z. B. wie folgt:

```
map.computeIfAbsent(key, (@NotNull String k) -> new ArrayList<>()).add(value);
map.computeIfAbsent(key, (@NotNull var k)    -> new ArrayList<>()).add(value);
```

Einen besonders langen, annotierten Typnamen können wir so durch *var* ersetzen – weglassen dürfen wir ihn nicht.

HTTP Client (Standard) – JEP 321 🌟

Java 11 ermöglicht mit der neuen Klasse *HttpClient*, sehr komfortabel auf HTTP-Schnittstellen zuzugreifen, z. B. wie folgt:

```
try (HttpClient client = HttpClient.newHttpClient()) {
  HttpRequest request = HttpRequest.newBuilder().uri(URI.create(url)).GET().build();
  HttpResponse<String> response = client.send(request, BodyHandlers.ofString());
  if (response.statusCode() == 200) { String responseBody = response.body(); . . . }
  else throw new IOException("HTTP error " + response.statusCode());
}
```

Neue Collection.toArray()-Methode 🌟

Um eine Collection in ein Array umzuwandeln, musste wir bisher der *toArray(...)*-Methode ein Array des Zieltyps übergeben. Ist dieses Array ausreichend groß, wird es mit den Werten der Collection gefüllt; andernfalls wird ein neues Array von gleichem Typ erzeugt. Seit Java 11 können wir der Methode auch eine Methodenreferenz auf den Konstruktor des Arrays übergeben, z. B. so:

```
String[] stringArray = stringList.toArray(String[]::new);
```

Neue String-Methoden 🌟

Die Klasse *String* wurde in Java 11 um die folgenden Methoden erweitert:

- *String.strip()*, *stripLeading()*, *stripTailing()* – entfernen aus einem String alle führenden und abschließenden bzw. nur die führenden oder nur die abschließenden Leerzeichen. Im Gegensatz zu *trim()*, das nur Zeichen entfernt, deren Codepoint U+0020 oder kleiner ist, werden by *strip()* alle im Unicode-Zeichensatz als „Whitespace" gekennzeichneten Zeichen entfernt.
- *String.isBlank()* – ergibt *true*, wenn ein String nur aus als „Whitespace" gekennzeichneten Zeichen besteht.
- *String.repeat(int count)* – erzeugt einen neuen String durch Wiederholung des Strings, auf dem die Methode aufgerufen wird.
- *String.lines()* – trennt einen String an allen Zeilenumbrüchen auf und liefert einen Stream aller Zeilen zurück.

Files.readString() und writeString() 🌟

Bisher waren mehrere Zeilen Java-Code (oder eine Drittanbieter-Bibliothek) nötig, um eine Datei in einen String zu lesen, bzw. einen String in eine Datei zu schreiben. Ab Java 11 gibt es dafür die Methoden *Files.writeString(...)* und *Files.readString(...)*.

Path.of() 🌟

Ab Java 11 kannst du ein *Path*-Objekt über die statische Default-Methode *of(...)* des *Path*-Interfaces erzeugen, z. B. so:

```
Path.of("foo/bar/baz");            // ← Relative path foo/bar/baz
Path.of("foo", "bar", "baz");      // ← Relative path foo/bar/baz
Path.of("/", "foo", "bar", "baz"); // ← Absolute path /foo/bar/baz
```

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 11 (Teil 2)

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Remove the Java EE and CORBA Modules – JEP 320 ⭐

In Java 11 wurden die folgenden Module aus dem JDK entfernt: *java.xml.ws (JAX-WS)*, *java.xml.bind (JAXB)*, *java.activation (JAF)*, *java.xml.ws.annotation (Common Annotations)*, *java.corba (CORBA)*, *java.transaction (JTA)*, *java.se.ee* (Aggregatormodul für die sechs zuvor genannten Module), *jdk.xml.ws* (Tools für JAX-WS) und *jdk.xml.bind* (Tools für JAXB).

Die genannten Technologien wurden ursprünglich für die Java-EE-Plattform entwickelt und wurden in Java 6 in die Standard Edition „Java SE" integriert. In Java 9 wurden sie als „deprecated" markiert, und in Java 11 werden sie endgültig entfernt.

Sollten dir diese Libraries beim Upgrade auf Java 11 fehlen, dann kannst du sie z. B. per Maven wieder in dein Projekt ziehen.

Flight Recorder – JEP 328 ⭐

Der Java Flight Recorder (kurz: JFR) kann während des Betriebs einer Anwendung zahlreiche JVM-Daten aufzeichnen und in einer Datei zur nachträglichen Analyse durch JDK Mission Control bereitstellen. JFR existiert bereits seit langem als kommerzielles Feature im Oracle-JDK. Ab Java 11 wird er Teil des OpenJDK und kann frei eingesetzt werden.

Epsilon: A No-Op Garbage Collector – JEP 318 ⭐

Der Epsilon Garbage Collector verwaltet – wie jeder andere GC – die Allokation von Objekten. Im Gegensatz zu allen anderen GCs gibt er die Objekte allerdings nie wieder frei. Das kann für Performance-Tests, extrem kurzlebige Applikationen und die vollständige Eliminierung von Latenzen hilfreich sein. Aktiviert wird Epsilon GC über die VM-Option *-XX:+UseEpsilonGC*.

Nest-Based Access Control – JEP 181

Die Java Language Specification (JLS) erlaubt seit jeher den Zugriff auf private Felder und Methoden von inneren Klassen. Da die Java Virtual Machine (JVM) das bisher *nicht* erlaubte, fügte der Java-Compiler (bis Java 10) beim Zugriff auf diese privaten Felder und Methoden sogenannte „synthetische Zugriffsmethoden" mit der Standard-Sichtbarkeit „package-private" ein. Demzufolge waren diese Felder und Methoden aus dem gesamten Paket erreichbar. Der Compiler gab eine entsprechende Warnung aus.

Im JDK 11 wurde die Zugriffskontrolle der JVM so erweitert, dass der Zugriff auf private Members von inneren Klassen ohne synthetische Zugriffsmethoden („synthetic accessors") auskommt.

Low-Overhead Heap Profiling – JEP 331

Ein wichtiges Debugging-Werkzeug sind Heap-Dumps zur Analyse der auf dem Heap liegenden Objekte. Entsprechende Tools konnten bisher allerdings nicht ermitteln, wo im Programmcode ein auf dem Heap liegendes Objekt erzeugt wurde.

In Java 11 wird das Java Virtual Machine Tool Interface (JVMTI) – also die Schnittstelle, über die die Tools Daten über die laufende Anwendung beziehen – um die Möglichkeit erweitert, Stack Traces aller Objekt-Allokationen zu sammeln. Heap-Analyse-Tools können diese Zusatzinformationen mit anzeigen und uns die Problemanalyse so wesentlich erleichtern.

Weitere Änderungen in Java 11

Dynamic Class-File Constants (JEP 309) – Das *.class*-Dateiformat wird um die Konstante *CONSTANT_Dynamic* erweitert, „die Sprachentwicklern und Compiler-Implementierern breitere Optionen für Ausdrucksfähigkeit und Leistung bietet."

Improve Aarch64 Intrinsics (JEP 315) – Intrinsics werden eingesetzt, um anstelle von Java-Code Architektur-spezifischen Assembler-Code auszuführen. Intrinsics für die AArch64-Plattform wurden verbessert, und neue Intrinsics wurden hinzugefügt.

Key Agreement with Curve25519 and Curve448 (JEP 324) – Schlüsselaustauschprotokolle mit elliptischen Kurven, wie „Curve25519" und „Curve448", ermöglichen eine besonders schnelle Ver- und Entschlüsselung von symmetrischen Schlüsseln.

Unicode 10 (JEP 327) – Der Unicode-Support wurde auf Version 10.0 angehoben, d. h. Klassen wie *String* und *Character* können mit den neu eingeführten Codeblöcken und Zeichen umgehen.

ChaCha20 and Poly1305 Cryptographic Algorithms (JEP 329) – Das JDK wird um die zwei genannten Kryptografiealgorithmen erweitert. Diese werden z. B. von Sicherheitsprotokollen wie TLS eingesetzt.

Transport Layer Security (TLS) 1.3 (JEP 332) – Neben Secure Socket Layer (SSL) 3.0, Transport Layer Security (TLS) 1.0 bis 1.2 und Datagram Transport Layer Security (DTLS) 1.0 und 1.2 unterstützt Java ab der Version 11 auch TLS 1.3.

ZGC: A Scalable Low-Latency Garbage Collector (Experimental) (JEP 333) – Ein neuer GC, der in Java 15 finalisiert wird.

Deprecate the Nashorn JavaScript Engine (JEP 335) – Die in Java 8 eingeführte JavaScript-Engine „Nashorn" wird in Java 11 als „deprecated for removal" markiert. Grund dafür sind die rasanten Entwicklungsgeschwindigkeiten von ECMAScript (dem Standard hinter JavaScript) und der *node.js*-Engine, die eine Weiterentwicklung von Nashorn zu aufwändig machen.

Deprecate the Pack200 Tools and API (JEP 336) – Das Kompressionsverfahren „Pack200" wurde in Java 5 eingeführt, um in den frühen 2000ern durch effektivere Kompression möglichst viel Bandbreite einzusparen. Die Entwicklungskosten stehen in Zeiten von 100-MBit-Leitungen in keinem Verhältnis zum Nutzen mehr. Daher wird Pack200 als „deprecated for removal" markiert.

 Copyright © 2025 HappyCoders.eu

Die neusten Features in Java 10

Mehr Details und die neueste Version dieser PDF-Sammlung findest du unter www.happycoders.eu/de/java-versionen.

Local-Variable Type Inference – JEP 268 🌟

Seit Java 10 können wir zur Deklaration lokaler Variablen („lokal" heißt: innerhalb von Methoden) das Keyword *var* verwenden, wie im Beispiel rechts.

Inwieweit man *var* verwendet, wird vermutlich zu heißen Diskussionen führen. Ich setze es dann ein, wenn es a) deutlich kürzer ist und ich b) den Datentyp klar im Code erkennen kann.

```
var i = 10;
var hello = "Hello world!";
var list = List.of(1, 2, 3, 4, 5);
var httpClient = HttpClient.newBuilder().build();
var status = getStatus();
var map = new HashMap<String, String>();
```

Immutable Collections 🌟

Mit *List.copyOf(...)*, *Set.copyOf(...)* und *Map.copyOf(...)* kannst du ab Java 10 unveränderliche Kopien von Listen, Sets und Maps erzeugen. Der Versuch auf diese Weise erzeugte Collections zu ändern führt zu einer *UnsupportedOperationException*.

Analog dazu kannst du mit *Collectors.toUnmodifiableList()*, *toUnmodifiableSet()* und *toUnmodifiableMap()* Streams in unveränderlichen Listen, Sets und Maps sammeln.

Optional.orElseThrow() 🌟

Die Methode *Optional.get()* wirft bei einem leeren *Optional* eine *NoSuchElementException*; daher sollte sie immer in Kombination mit *Optional.isPresent()* genutzt werden. IDEs und statische Codeanalysetools erzeugen eine Warnung, wenn man das nicht tut. Sollte solch eine Exception hingegen gewünscht sein, stört diese Warnung.

Deshalb wurde die Methode *Optional.orElseThrow()* eingeführt, die exakt das gleiche tut wie *Optional.get()*, jedoch von statischen Codeanalysetools nicht moniert wird, da ihr Name das Verhalten eindeutig beschreibt.

Parallel Full GC for G1 – JEP 307

Der Garbage-First (G1) Garbage Collector hat in Java 9 den Parallel Collector als Standard-GC abgelöst. Während der Parallel GC auch eine vollständige Gargage Collection („Full GC") parallel zur laufenden Applikation durchführen konnte, war dies beim G1 bisher nicht möglich – er musste dazu die Anwendung anhalten („stop-the-world"), was zu spürbaren Latenzen führen konnte.

In Java 10 wurde nun auch die vollständige Gargage Collection des G1 Collectors parallelisiert.

Application Class-Data Sharing – JEP 310

Class-Data Sharing (kurz: CDS) bedeutet, dass einmal geladene *.class*-Dateien in einer architekturspezifischen binären Form gecacht werden, so dass der JVM-Start beschleunigt und durch gemeinsamen Zugriff auf diesen Cache von ggf. mehreren parallel laufenden JVMs aus der Gesamt-Speicherbedarf reduziert wird.

In Java 10 wurde CDS so erweitert, dass nicht mehr nur Klassen der JDK-Klassenbibliothek gecacht werden können, sondern auch die Klassen der Anwendung („Application CDS", kurz „AppCDS").

Weitere Änderungen in Java 10

Consolidate the JDK Forest into a Single Repository (JEP 296) – Der JDK-Quellcode, der bisher in acht separaten Repositories lag, wurde in ein Monorepo konsolidiert, was durch atomare Commits, Branches und PRs die Arbeit am JDK stark vereinfacht.

Garbage-Collector Interface (JEP 304) – Im JDK-Quellcode wurde eine saubere Garbage-Collector-Schnittstelle eingeführt, um die Algorithmen der GCs vom Interpreter und den Compilern zu isolieren und die Entwicklung neuer GCs zu vereinfachen.

Thread-Local Handshakes (JEP 312) – Thread-Local Handshakes sind eine Optimierung, um die Leistung der virtuellen Maschine auf x64- und SPARC-basierten Architekturen zu verbessern.

Remove the Native-Header Generation Tool (javah) (JEP 313) – Mit dem Tool *javah* konnte man native Header-Dateien für JNI erzeugen. Die Funktionalität wurde in Java 10 in den Java-Compiler *javac* integriert.

Additional Unicode Language-Tag Extensions (JEP 314) – Sogenannte „Language-Tag Extensions" ermöglichen es, in einem *Locale*-Objekt Zusatzinformationen über die Währung, den ersten Tag der Woche, die Region und die Zeitzone zu hinterlegen.

Heap Allocation on Alternative Memory Devices (JEP 316) – Der Java-Heap kann ab Java 10 – statt auf herkömmlichem RAM – auch auf einem alternativen Speichergerät, z. B. auf NV-DIMM (nicht-flüchtigem Speicher) angelegt werden.

Experimental Java-Based JIT Compiler (JEP 317) – Seit Java 9 wird der Graal Compiler (ein in Java geschriebener Java-Compiler) als experimenteller Ahead-of-Time (AOT)-Compiler mit ausgeliefert. Ab Java 10 kann Graal auch als Just-in-Time (JIT)-Compiler eingesetzt werden – zunächst auf der Linux/x64-Plattform.

Root Certificates (JEP 319) – Bisher nur im Oracle-JDK enthaltene Root-Zertifikate wurden in das OpenJDK übernommen.

Time-Based Release Versioning (JEP 322) – Versionsangaben enthalten ab sofort das Release-Datum sowie die Angabe, ob es sich um ein Long-Term-Support-Release handelt. Beispiele: *java version "10.0.2" 2018-07-17*, *java version "11.0.11" 2021-04-20 LTS*

 Copyright © 2025 HappyCoders.eu